

+ 26

In Pursuit of a 6,000 FPS Game Boy Emulator

TOM TESCH



20
26



About me

- Teacher @ Howest University of Applied Sciences within the “Digital Arts and Entertainment” bachelor: Game Development major
 - C++
 - Algorithms
 - Computational System Fundamentals
 - Audio Programming
 - Retro Console Programming
- <https://www.howest.be/en>
- <https://www.digitalartsandentertainment.be/>
- tom.tesch@howest.be
- <https://www.linkedin.com/in/tom-tesch/>

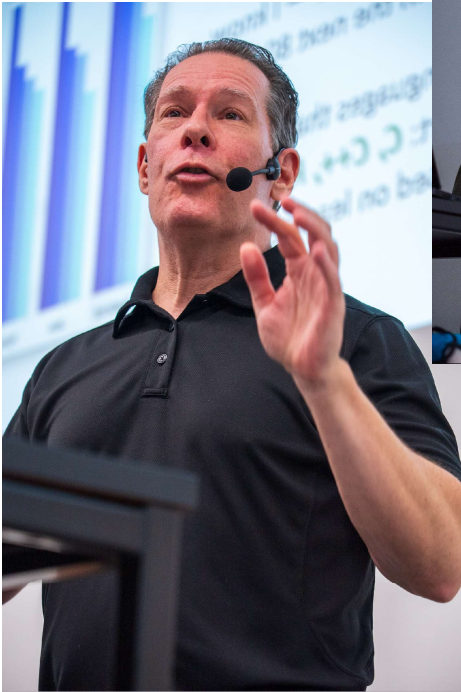




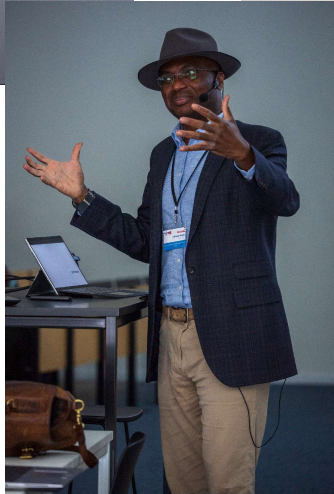
<https://becpp.org/blog/>



<https://becpp.org/Symposium2026/>

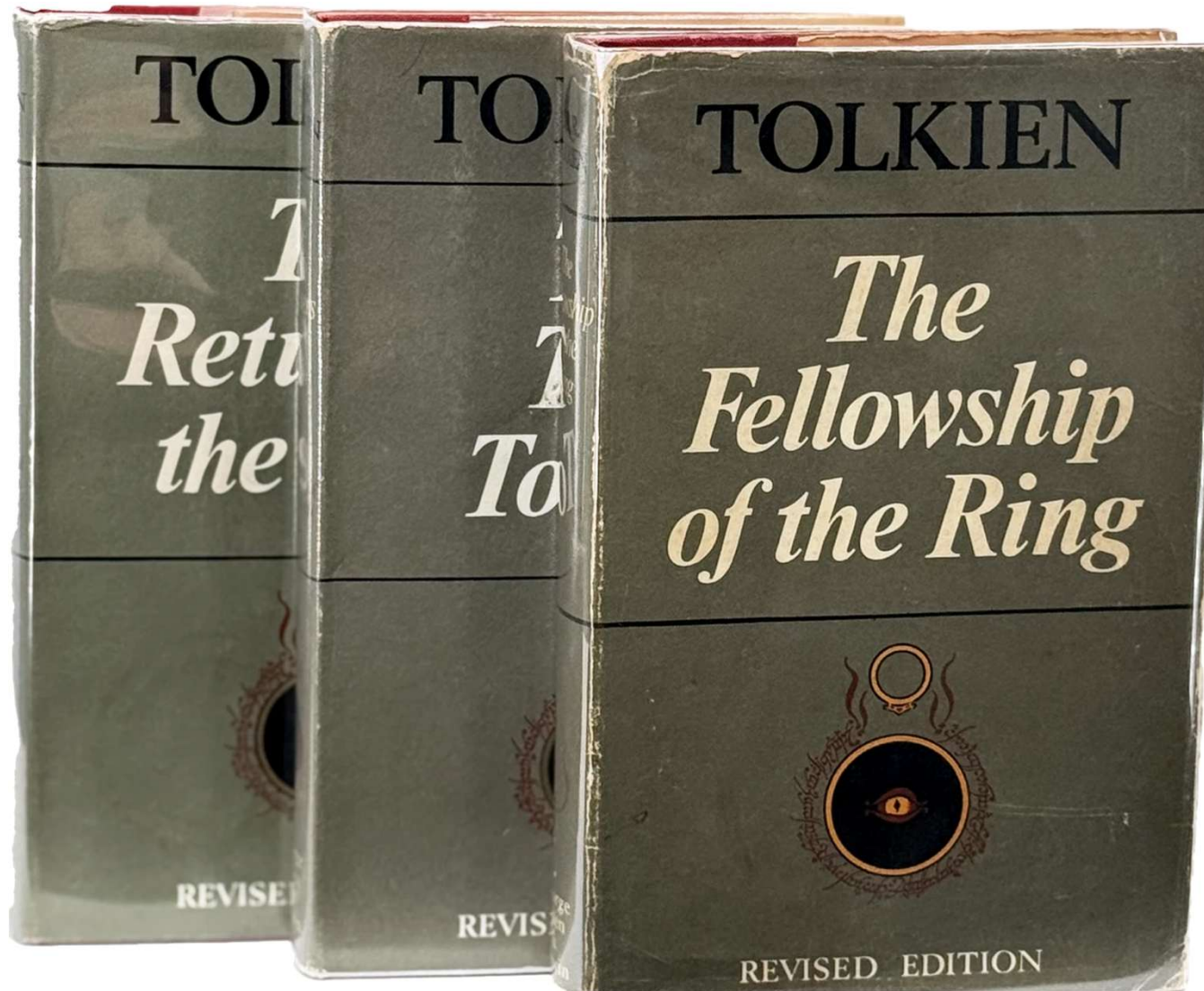


Tom Tesch



In Pursuit of a 6,000 FPS Game Boy Emulator





An apology to start with...

In Pursuit of a 6,000 FPS Game Boy Emulator

should actually be

In Pursuit of a 5,973 FPS Game Boy Emulator

I wanted a 100x speedup

The original **Game Boy DMG-001** refreshes at about **59.73 Hz**.

More precisely:

$$\frac{4,194,304 \text{ cycles/s}}{70,224 \text{ cycles/frame}} = 59.7275 \text{ frames/s}$$

Source: <https://mgba-emu.github.io/gbdoc/>

<https://youtu.be/ScmhRNSrRP4>

Blazing Trails: Building the World's Fastest GameBoy Emulator in Modern C++

TOM TESCH

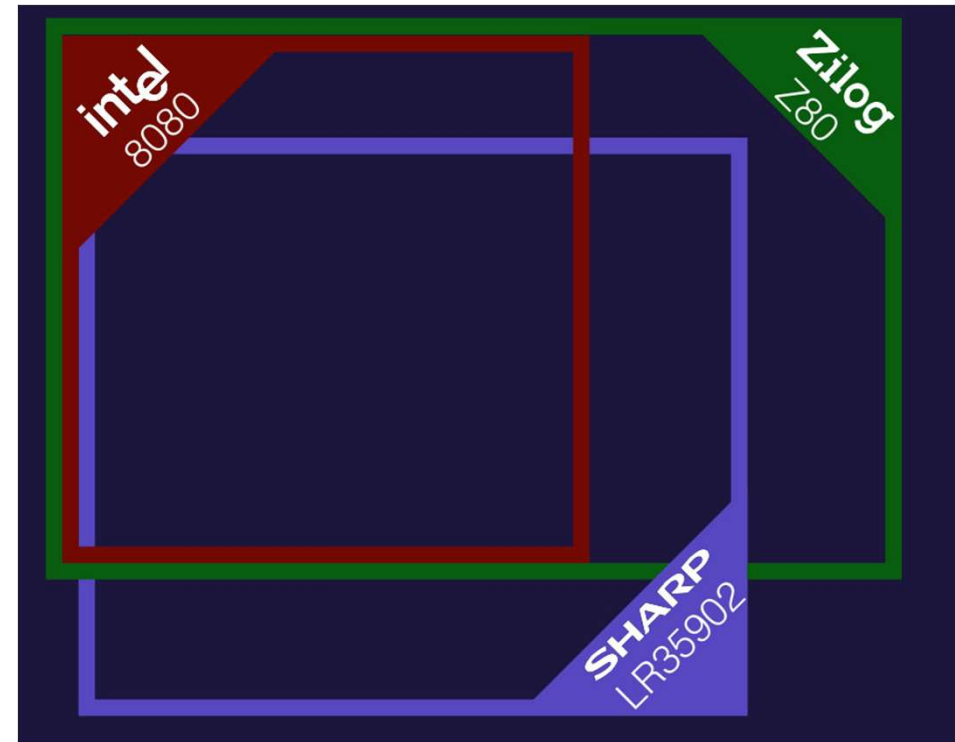


20
24



CPU

- System On a Chip (SoC)
- DMG-CPU or Sharp LR35902
- Sharp SM83 (mix between Z80 and Intel 8080)
- 4,194,304 Hertz $\approx$ 4 Mhz
- No IO ports = completely memory mapped
- Only 8080's set of registers
- Z80's extended bit manipulation instruction set
- Some additional new instructions



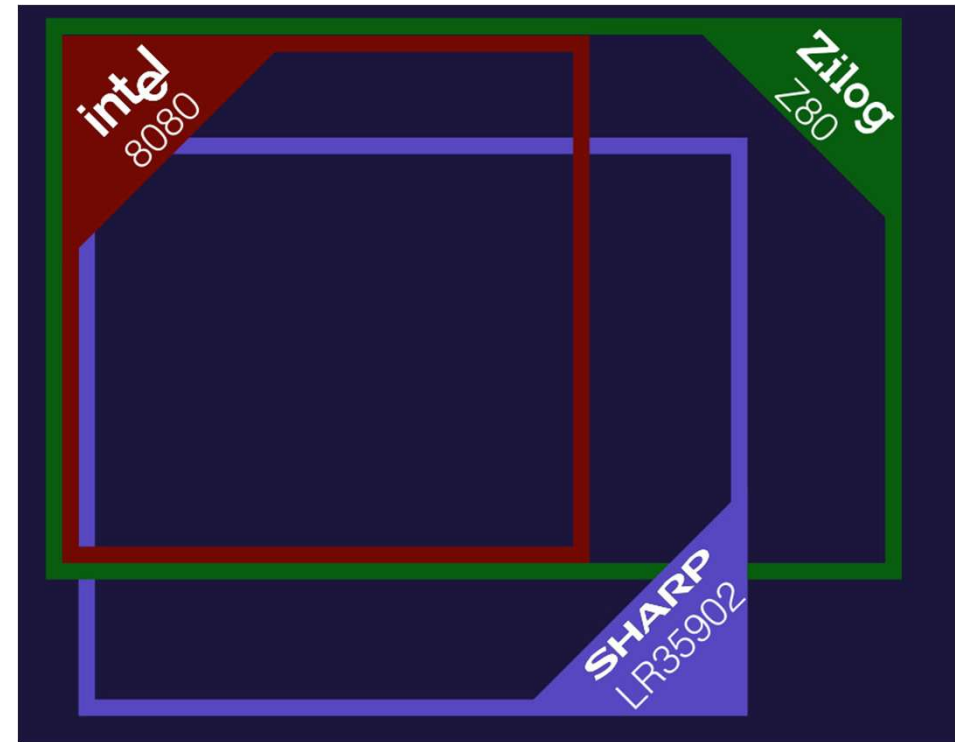
The ultimate Game Boy talk – Michael Steil



https://media.ccc.de/v/33c3-8029-the_ultimate_game_boy_talk

CPU

- System On a Chip (SoC)
- DMG-CPU or Sharp LR35902
- Sharp SM83 (mix between Z80 and Intel 8080)
- **4,194,304 Hertz $\approx$ 4 Mhz**
- No IO ports = completely memory mapped
- Only 8080's set of registers
- Z80's extended bit manipulation instruction set
- Some additional new instructions



T-Cycle vs M-Cycle

- **T-Cycle:** Tick Cycle (also called Machine Cycle in some contexts). A T-cycle represents the smallest unit of time in the Game Boy's CPU, typically corresponding to one clock tick of the CPU.
Emulators with T-cycle accuracy simulate the exact number of clock ticks for every instruction, providing the highest level of timing precision.
- **M-Cycle:** Memory Cycle. An M-cycle represents a higher-level unit of time used by the Game Boy's CPU for executing instructions. Each instruction takes a specific number of M-cycles, with each **M-cycle typically equating to 4 T-cycles.**

T-Cycle vs M-Cycle



- **T-Cycle:** Tick Cycle (also called Machine Cycle in some contexts). A T-cycle represents the smallest unit of time in the Game Boy's CPU, typically corresponding to one clock tick of the CPU. **Emulators with T-cycle accuracy simulate the exact number of clock ticks for every instruction, providing the highest level of timing precision.**
- **M-Cycle:** Memory Cycle. An M-cycle represents a higher-level unit of time used by the Game Boy's CPU for executing instructions. Each instruction takes a specific number of M-cycles, with each **M-cycle typically equating to 4 T-cycles.**

Since Tetris is an early (and thus simple) game

The DMG-01 runs at
 $2^{20} = 1.048576 \text{ MHz}$

Aside

16

Aside

The screenshot shows the 6502 emulator interface with the following components:

- Registers / Flags:** A row of nine boxes representing the 6502 registers and flags. The values are: A=00, B=00, C=00, D=00, IP=0F (highlighted in blue), SP=E7 (highlighted in orange), Z=FALSE, C=FALSE, and F=FALSE.
- RAM:** A grid of 256 bytes (16 rows by 16 columns). The first row (addresses 00-0F) is highlighted in yellow. The first column (addresses 00-0F) is highlighted in cyan. The value at address 000F is 06 (highlighted in blue). The value at address 000E is 03 (highlighted in orange).
- Instructions:** A dashed line separates the RAM from the instructions section. An arrow points down to the instructions area.
- Stack Display:** A dashed line separates the RAM from the stack display section. An arrow points up to the stack display area. The stack display shows the address 000E (highlighted in orange) and the value 03 (highlighted in orange).

```
1F = .loop
```

```
0F = .start
```

“Hello World!” + 0 terminator

Rest of instructions

“Hardware”

- Eight-bit controller
- 256 bytes of RAM – 24 bytes used for character display
- 4 general purpose registers (**A**, **B**, **C** & **D**)
- Stack pointer (**SP**)
- Instruction pointer (**IP**)
- Three flags (**Z**ero, **C**arry and **F**ault)
- Simulator runs up to a blazing 16Hz

Building the World's Fastest GameBoy Emulator in Modern C++

Tom Tesch



JETBRAINS

Optiver 

Aside

Three nested loops for long execution

The screenshot displays a Game Boy emulator interface. On the left, the 'Code (Instruction Set)' panel shows the following assembly code:

```
MOV A, 0
MOV B, stop
MOV C, 0
MOV D, 0
loop1: INC A
loop2: INC C
loop3: INC D
CMP D, [B]
JNZ loop3
CMP C, [B]
JNZ loop2
CMP A, [B]
JNZ loop1
stop: HLT
```

On the right, the 'Output' panel shows a progress bar. Below it, the 'CPU & Memory' panel displays the 'Registers / Flags' section:

A	B	C	D	IP	SP	Z	C	F
00	00	00	00	00	E7	FALSE	FALSE	FALSE

Below the registers, the 'RAM' section shows a memory dump with hexadecimal values. At the bottom, the 'Clock speed' is set to 4 HZ, and the 'Register addressing' section shows 'A: Show B: Show C: Show D: Show'.

On my 3.2 GHz machine:

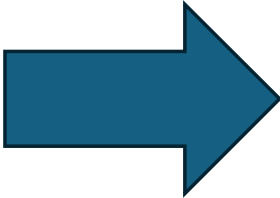
50529029 instructions
in 200milliseconds

→ 250MIPS

Three nested loops for long execution

Code (Instruction Set)

```
MOV A, 0
MOV B, stop
MOV C, 0
MOV D, 0
loop1: INC A
loop2: INC C
loop3: INC D
      CMP D, [B]
      JNZ loop3
      CMP C, [B]
      JNZ loop2
      CMP A, [B]
      JNZ loop1
stop: HLT
```



```
--asm {
    mov al, 0
    mov ebx, offset zero
    mov cl, 0
    mov dl, 0
loop1:
    inc al
loop2:
    inc cl
loop3:
    inc dl
    cmp dl, [ebx]
    jne loop3
    cmp cl, [ebx]
    jne loop2
    cmp al, [ebx]
    jne loop1
}
```



On my 3.2 GHz machine:

50529029 instructions
in 200milliseconds

→ 250MIPS

Aside

```
__asm {
    mov al, 0
    mov ebx, offset zero
    mov cl, 0
    mov dl, 0
loop1:
    inc al
loop2:
    inc cl
loop3:
    inc dl
    cmp dl, [ebx]
    jne loop3
    cmp cl, [ebx]
    jne loop2
    cmp al, [ebx]
    jne loop1
}
```

```
#include <chrono>
#include <iostream>

int main() {
    // Must be static so MSVC __asm can use 'offset zero'
    static unsigned char zero{};

    auto start{std::chrono::high_resolution_clock::now()};
    __asm { ... }
    auto end{std::chrono::high_resolution_clock::now()};

    auto elapsed_us{
        std::chrono::duration_cast<std::chrono::microseconds>(end - start)
        .count()};

    constexpr auto instruction_count{50'529'029.0};
    auto elapsed_seconds{static_cast<double>(elapsed_us) / 1'000'000.0};
    auto mips{instruction_count / (elapsed_seconds * 1e6)};

    std::cout << "Executed " << instruction_count << " instructions in "
               << elapsed_us << " microseconds.\n";
    std::cout << "MIPS: " << mips << "\n";

    return 0;
}
```



```

__asm {
    mov al, 0
    mov ebx, offset zero
    mov cl, 0
    mov dl, 0
loop1:
    inc al
loop2:
    inc cl
loop3:
    inc dl
    cmp dl, [ebx]
    jne loop3
    cmp cl, [ebx]
    jne loop2
    cmp al, [ebx]
    jne loop1
}

```

**“Only” 40x
slower**

Tom Tesch

```

Executed 5.0529e+07 instructions in 4872 microseconds.
MIPS: 10371.3

```

```

#include <chrono>
#include <iostream>

int main() {
    // Must be static so MSVC __asm can use 'offset zero'
    static unsigned char zero{};

    auto start{std::chrono::high_resolution_clock::now()};
    __asm { ... }
    auto end{std::chrono::high_resolution_clock::now()};

    auto elapsed_us{
        std::chrono::duration_cast<std::chrono::microseconds>(end - start)
        .count()};

    constexpr auto instruction_count{50'529'029.0};
    auto elapsed_seconds{static_cast<double>(elapsed_us) / 1'000'000.0};
    auto mips{instruction_count / (elapsed_seconds * 1e6)};

    std::cout << "Executed " << instruction_count << " instructions in "
               << elapsed_us << " microseconds.\n";
    std::cout << "MIPS: " << mips << "\n";

    return 0;
}

```

In Pursuit of a 6,000 FPS Game Boy Emulator

23

Oversimplified conclusion

- My PC boosts up to 4 Ghz
- The Game Boy runs at 1 Mhz
- Emulation is 40x slower than original code

∴ I should be able to emulate the DMG-01 at **100x**





Zero Cost Scripting Languages for Game Engines With C++26 Static Reflection

Monday September 14, 2026 16:45 - 17:45 MDT

Willow Lake 3/4/5

Scripting in a C++ game engine should not cost you the engine's native performance. This talk shows how C++26 static reflection and `std::meta::substitute` in particular can lift a scripting language's bytecode into C++ template structures that the compiler optimizes away entirely, collapsing the interpreter dispatch loop into the same machine code you'd write by hand. Scripting languages like AngelScript and Lua are invaluable in C++ engines: they give designers a fast iteration loop without rebuilding the engine. But they come with a paradox. The engine you chose for raw performance now spends cycles on every frame interpreting a slower language, juggling a software stack and checking types at runtime. We will walk through the technique step by step, starting from a plain bytecode interpreter and ending at a fully reflected program where a scripted `sum(0..10)` compiles to `mov eax, 45; ret`. Along the way you'll learn the core C++26 reflection primitives (`^^`, `[: :]`, `std::meta::substitute`), how to assemble bytecode into structural templates like `block<>` and `loop<>`, and how these techniques generalize to embedding any stack or register based scripting language in your engine with zero runtime overhead.

Presenters

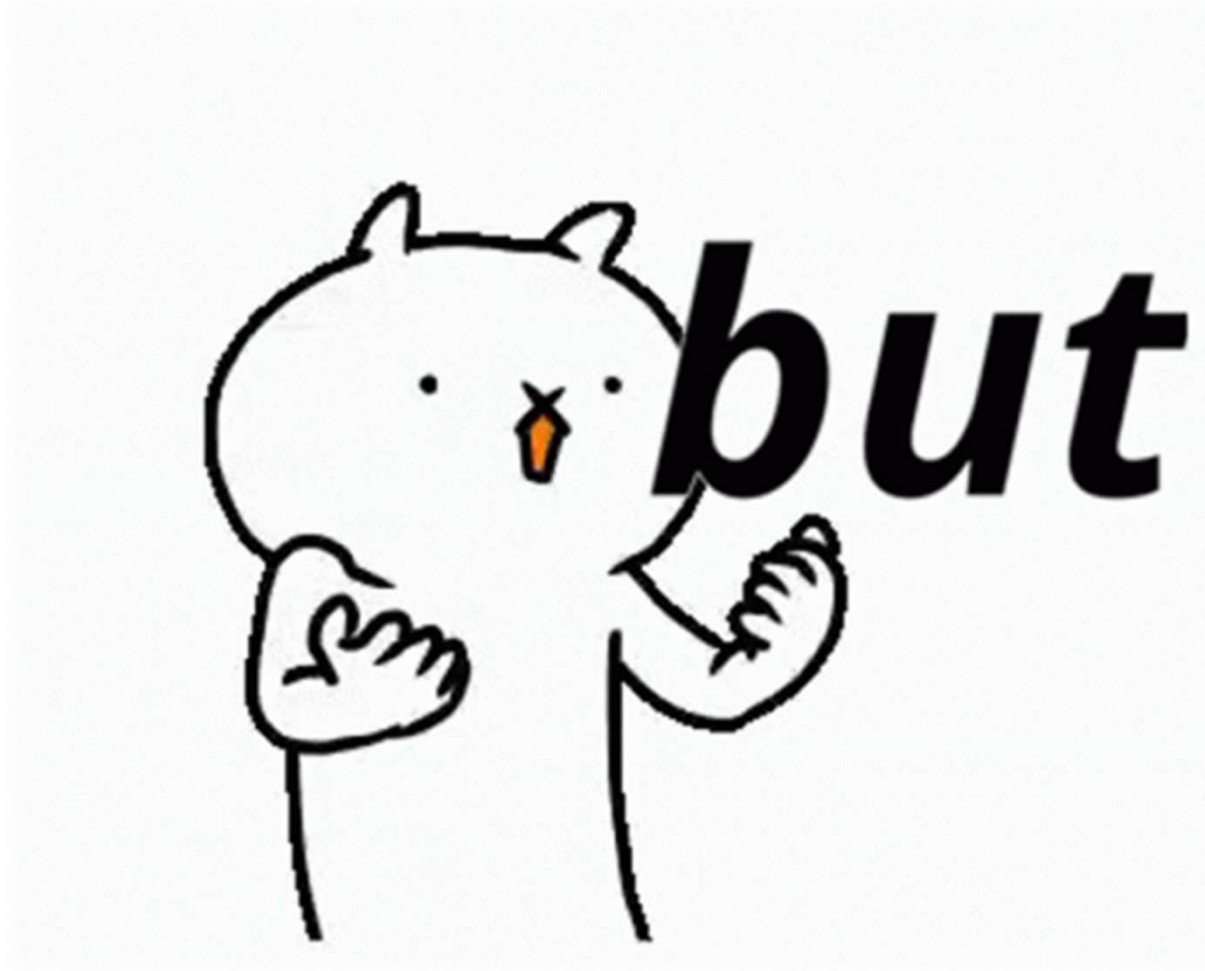


Koen Samyn

Koen is a senior software engineer and lecturer at digital art and entertainment whose primary focus is bringing modern C++ to GPU-driven game technology. Over the past decade he has built and optimized compute-shader pipelines for lighting, physics, and inverse kinematics. In the... [Read More →](#)

GameDev

GameBoy Emulation with Tetris in Modern C++



Writing an emulator is a great way to learn
how computers work...



Back to Basics: Computer Systems

📅 Thursday September 17, 2026 16:45 - 17:45 MDT

📍 Red Rock 8/9

Every software engineer needs to understand the fundamentals of how your computer works. My mantra for teaching computer systems is 'no more magic', and in this talk I'll do my best to dispel what feels like 'magic' in our hardware in a single hour. In this talk, we will investigate '3' pictures of the 'the compilation process', 'program stack', and a 'cpu' to understand how these components work together to ultimately compile and execute code. An emphasis during this talk will focus on the 'CPU' and the hardware components (e.g. registers, cache memory, main memory, page table) interact with the operating system to efficiently execute a program. Hardware will be presented side-by-side with code so we can understand what happens in the hardware as software executes. Attendees will leave this talk with a better mental model of how their hardware works, and the implications their software design choices have on performance.

Presenters



Mike Shah

Professor / (occasional) 3D Graphics Engineer

Mike Shah is currently a teaching faculty at Yale University with primary teaching interests in computer systems, computer graphics, and game engines. Mike's research interests are related to performance engineering (dynamic analysis), software visualization, and computer graphics... [Read More →](#)

● [Back to Basics](#)

Subject [Software Design](#), Tags [Breakouts](#)



The Biggest Misconception of Computer Science

📅 Thursday September 17, 2026 15:15 - 16:15 MDT

📍 Willow Lake 3/4/5

From our very first algorithms class, we are taught a simple rule: a better Big-O complexity means a faster algorithm. We spend years mastering asymptotic analysis, memorizing complexity tables, and losing sleep over worst-case scenarios. Big-O becomes a mental shortcut for “good” and “bad” code.

But the real world doesn’t run on whiteboards.

Modern performance is shaped far more by hardware realities than by asymptotic notation alone. CPUs have deep cache hierarchies, wide vector units, speculative execution, and memory systems that punish the “theoretically optimal” solution. GPUs thrive on massive parallelism where simple linear work can outperform asymptotically superior algorithms. Even on regular CPUs, cache-friendly linear scans often beat clever sub-linear approaches that fight memory latency.

In this talk, we will challenge the traditional Big-O mindset. We’ll look at classic algorithms through a modern lens and explore how hardware-aware designs cache-efficient layouts, SIMD/AVX vectorization, and parallel execution models can outperform algorithms with “worse” theoretical complexity. You’ll see why a higher Big-O algorithm can be faster, more scalable, and more predictable in practice.

The goal is not to dismiss Big-O, but to put it back in its proper place: as a tool, not a truth. By the end of this talk, you’ll think differently about performance and start writing code that works with the hardware, not against it.



The Biggest Misconception of Computer Science

📅 Thursday September 17, 2026 15:15 - 16:15 MDT

📍 Willow Lake 3/4/5

From our very first algorithms class, we are taught a simple rule: a better Big-O complexity means a faster algorithm. We spend years mastering asymptotic analysis, memorizing complexity tables, and looking down our noses at

Presenters



Alex Dathskovsky

Director of SW engineering, Speeddata

Alex has over 18 years of software development experience, working on systems, low-level generic tools and high-level applications. Alex has worked as an integration/software developer at Elbit, senior software developer at Rafael, technical leader at Axxana, Software manager at Abbott... [Read More →](#)

● General

Subject **Algorithms, Concurrency, Future C++, Math/Analysis, Software Design, Software Quality,** Tags **Breakouts**

talk, you'll think differently about performance and start writing code that works with the hardware, not against it.



The Address is Not The Place: Object Residency in C++26

📅 Thursday September 17, 2026 10:30 - 12:00 MDT

📍 Colorado A

Random Access Memory is what you buy when you don't know what your program will do next.

Until recently, alternative tiers of memory with vastly different latency characteristics were largely speculative. Then DRAM got expensive. CXL memory pools, far memory, even high bandwidth flash moved from blips to serious contenders. C++ has grown its vocabulary for ownership (smart pointers) and lifetimes (RAII); but intelligent memory tiering, specifically object residency, is a word we don't natively have.

Attempts have been made to solve this issue at an OS-level (Meta's TPP), or even a compiler/runtime level (OBASE) but they approach the problem transparently by design. And thus...blind by design. You can improve tiered object locality with clever placement within pages, but all these methods are structurally incapable of predicting intent. What if the power was in the developer's hands?

With reflection (P2996) and annotations (P3394), C++26 finally lets us extend the vocabulary ourselves. Game developers have deeply understood smart data grouping for decades, but much of the work is locked behind game engines; not suitable for general application development. `[[likely]]` and `[[unlikely]]` hint hot and cold paths, but the future of high-performance development hints at general purpose object residency to tier-aware allocators.

This talk introduces an open-source C++26 residency library, demonstrating capabilities against current, and even (simulated) future hardware. The pieces are in place for C++ to create durable abstractions for object residency; with the flexibility to accommodate memory tiers that have not been invented yet!



The Address is Not The Place: Object Residency in C++26

📅 Thursday September 17, 2026 10:30 - 12:00 MDT

📍 Colorado A

Random Access Memory is what you buy when you don't know what your program will do next.

Presenters



Laurie Kirk

Researcher, Google

Laurie Kirk is a researcher at Google specializing in C++, reverse engineering, and deobfuscation. She runs a YouTube channel (@LaurieWired) that covers all sorts of in-depth research topics on reverse engineering, programming, and software optimizations. She has spoken at multiple... [Read More →](#)

● Plenary

Tags [Plenary](#)

performance development hints at general purpose object residency to tier-aware allocators.

This talk introduces an open-source C++26 residency library, demonstrating capabilities against current, and even (simulated) future hardware. The pieces are in place for C++ to create durable abstractions for object residency; with the flexibility to accommodate memory tiers that have not been invented yet!

“When you are building
an emulator, you can
play games and
~~pretend~~ explain that
you are working”

Matt Godbolt

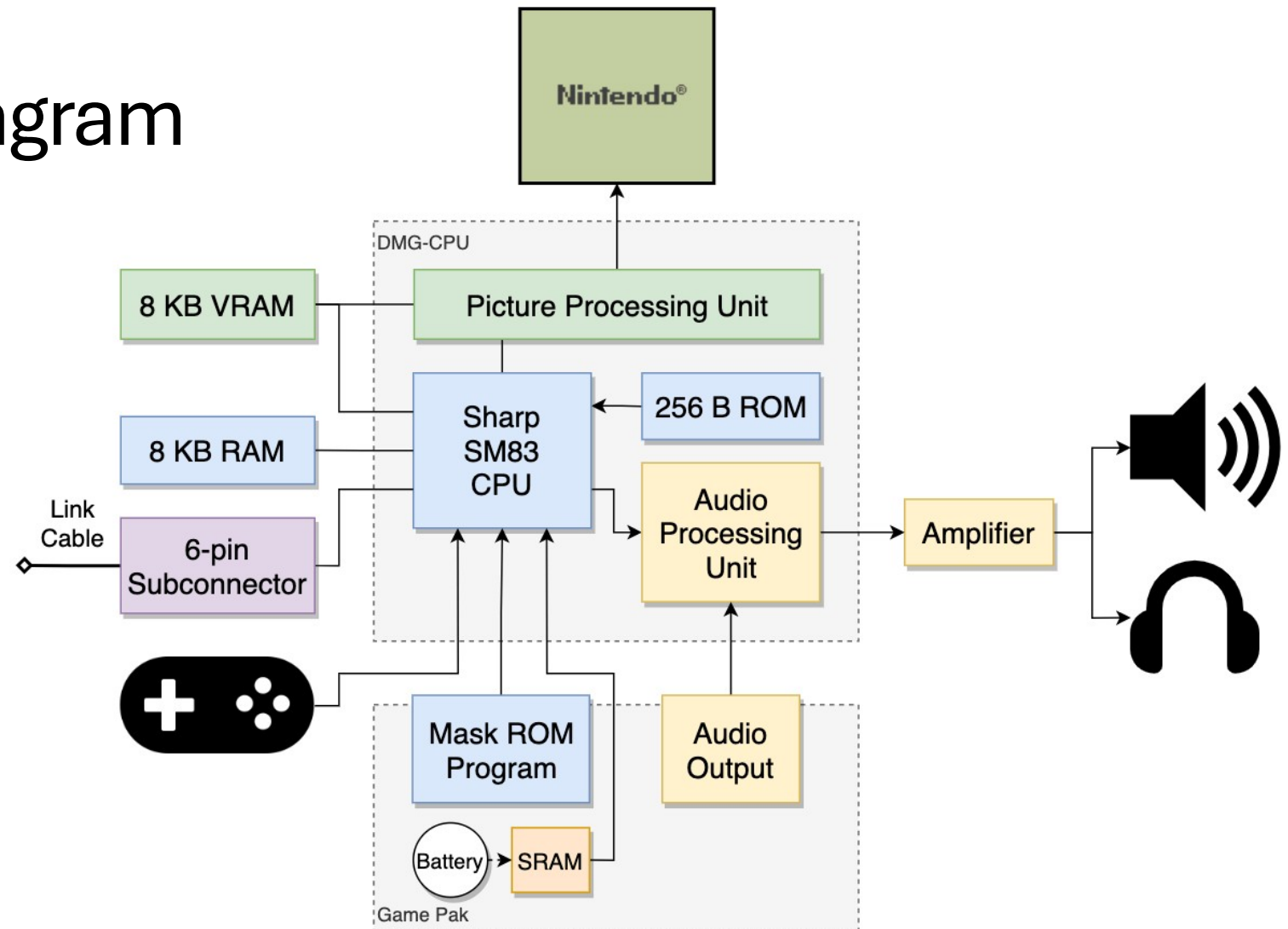


Why the Game Boy?

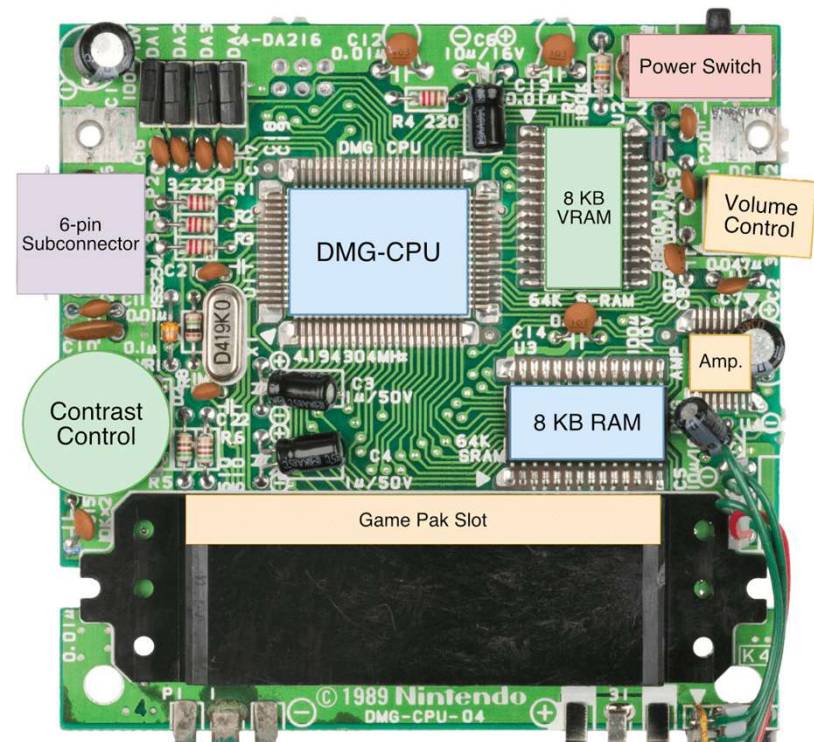
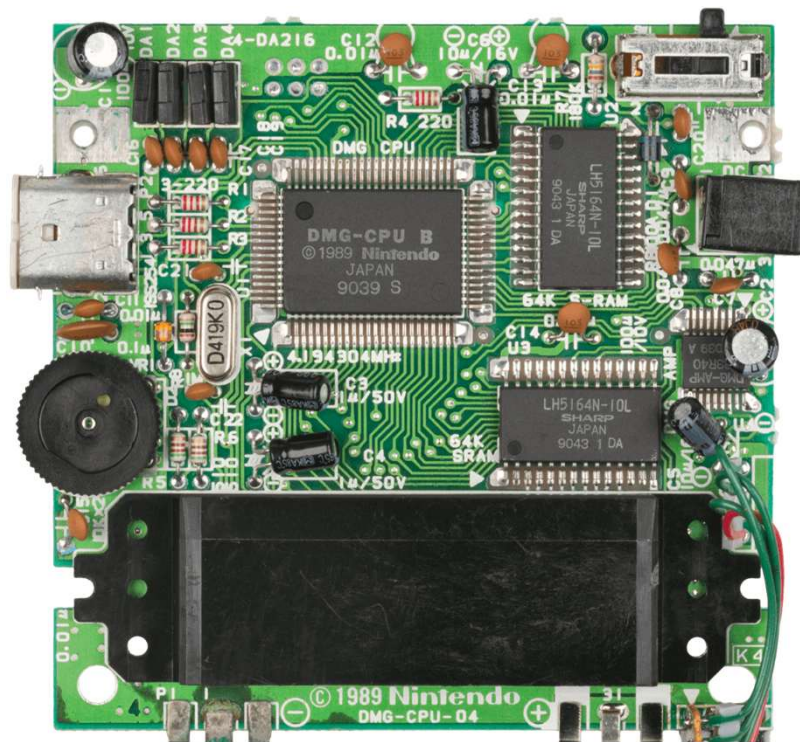




CPU Block Diagram



Motherboard (revision 04)



Memory & Memory mapped access

- 8 KiB of RAM (called WRAM)
 - Four times more than the NES!
- 8 KiB of Video RAM (called VRAM)
- Memory map:
 - Cartridge space.
 - WRAM and Display RAM.
 - I/O (joypad, audio, graphics and LCD)
 - Interrupt controls.

Interrupt Register	0xFFFF
High RAM	0xFF80 - 0xFFFE
Unusable	0xFF4C - 0xFF7F
I/O	0xFF00 - 0xFF4B
Unusable	0xFE00 - 0xFEFF
Sprite Attributes	0xFE00 - 0xFE9F
Unusable	0xE000 - 0xFDEF
Internal RAM	0xC000 - 0xDFFF
Switchable RAM Bank	0xA000 - 0xBFFF
Video RAM	0x8000 - 0x9FFF
Switchable ROM Bank	0x4000 - 0x7FFF
ROM	0x0000 - 0x3FFF

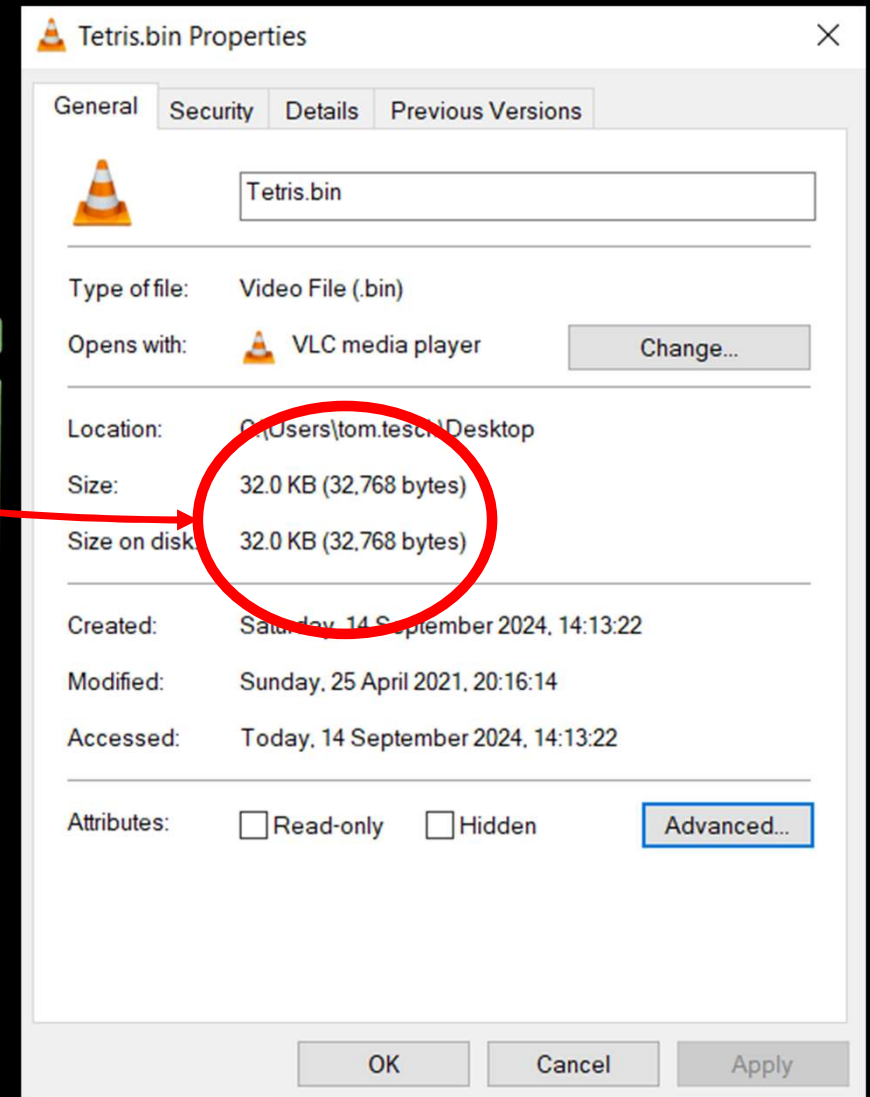
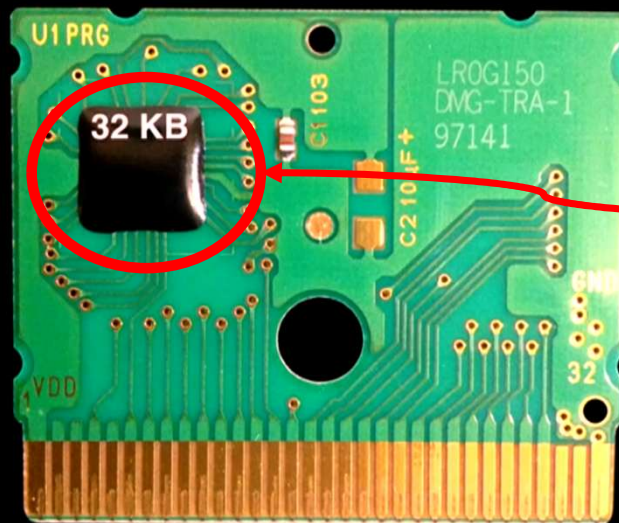
Why Tetris?



But really: why tetris?

- M-cycle precision emulation is sufficient
- Only 284 of the 501 opcodes are used (and need to be implemented)
- Limited size means no memory banking – no MBC needed

No MBC needed!



Does not require audio to play...

Tetris is Hard, Even to Approximate

Erik D. Demaine* Susan Hohenberger* David Liben-Nowell*

February 1, 2008

Abstract

In the popular computer game of *Tetris*, the player is given a sequence of tetromino pieces and must pack them into a rectangular gameboard initially occupied by a given configuration of filled squares; any completely filled row of the gameboard is cleared and all pieces above it drop by one row. We prove that in the offline version of Tetris, it is **NP**-complete to maximize the number of cleared rows, maximize the number of tetrises (quadruples of rows simultaneously filled and cleared), minimize the maximum height of an occupied square, or maximize the number of pieces placed before the game ends. We furthermore show the extreme inapproximability of the first and last of these objectives to within a factor of $p^{1-\varepsilon}$, when given a sequence of p pieces, and the inapproximability of the third objective to within a factor of $2 - \varepsilon$, for any $\varepsilon > 0$. Our results hold under several variations on the rules of Tetris, including different models of rotation, limitations on player agility, and restricted piece sets.

Tetris is Hard, Even to Approximate

Erik D. Demaine*

Susan Hohenberger*

David Liben-Nowell

February 1, 2008

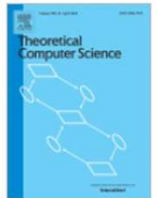
Abstract

In the popular computer game of *Tetris*, the player is given a sequence of tetrominoes and must pack them into a rectangular gameboard initially occupied by a given configuration of filled squares; any completely filled row of the gameboard is cleared and all pieces above it fall by one row. We prove that in the offline version of Tetris, it is NP-complete to maximize the number of cleared rows, maximize the number of tetrises (quadruples of rows simultaneously filled and cleared), minimize the maximum height of an occupied square, or maximize the number of pieces placed before the game ends. We furthermore show the extreme inapproximability of the first and last of these objectives to within a factor of $p^{1-\epsilon}$, when given a sequence of p pieces, and the inapproximability of the third objective to within a factor of $2 - \epsilon$, for any $\epsilon > 0$. Our results hold under several variations on the rules of Tetris, including different rotation rules, limitations on player agility, and restricted piece sets.



Theoretical Computer Science

Volume 992, 21 April 2024, 114405



How fast can we play Tetris greedily with rectangular pieces? ☆

Justin Dallant¹  , John Iacono² 

Tetris is Hard, Even to Approximate

Erik D. Demaine*

Susan Hohenberger*

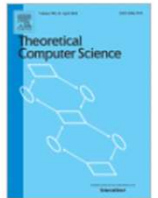
David Liben-Nowell

February 1, 2008



Theoretical Computer Science

Volume 992, 21 April 2024, 114405



Abstract

In the popular computer game of *Tetris*, the player is given a sequence of tetrominoes and must pack them into a rectangular gameboard initially occupied by a given configuration of filled squares; any complete row is cleared, and the player receives a score for each row cleared. We prove that, for any number of cleared rows, minimizing the number of pieces placed before the first and last of these rows cleared is NP-hard. Our results hold under several variations, including no rotation, limitations on play, and a “hard drops only” mode.

Tetris with Few Piece Types

MIT Hardness Group*

Erik D. Demaine[†]

Holden Hall[†]

Jeffery Li[†]

Abstract

We prove NP-hardness and #P-hardness of Tetris clearing (clearing an initial board using a given sequence of pieces) with the Super Rotation System (SRS), even when the pieces are limited to *any two* of the seven Tetris piece types. This result is the first advance on a question posed twenty years ago: which piece sets are easy vs. hard? All previous Tetris NP-hardness proofs used five of the seven piece types. We also prove ASP-completeness of Tetris clearing, using three piece types, as well as versions of 3-Partition and Numerical 3-Dimensional Matching where all input integers are distinct. Finally, we prove NP-hardness of Tetris survival and clearing under the “hard drops only” and “20G” modes, using two piece types, improving on a previous “hard drops only” result that used five piece types.

Play Tetris Angular

Tetris is Hard, Even to Approximate

Erik D. Demaine* Susan Hohenberger* David Liben-Nowell

February 1, 2020



Tetris is NP-hard even with $O(1)$ Rows or Columns

SUALEH ASIF^{1,a)} MICHAEL COULOMBE^{1,b)} ERIK D. DEMAINE^{1,c)} MARTIN L. DEMAINE^{1,d)}
ADAM HESTERBERG^{1,e)} JAYSON LYNCH^{1,f)} MIHIR SINGHAL^{1,g)}

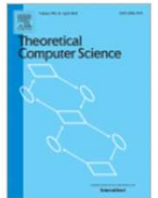
Received: January 6, 2020, Accepted: June 1, 2020

Abstract: We prove that the classic falling-block video game *Tetris* (both survival and board clearing) remains NP-complete even when restricted to 8 columns, or to 4 rows, settling open problems posed over 15 years ago. Our reduction is from 3-PARTITION, similar to the previous reduction for unrestricted board sizes, but with a better packing of buckets. On the positive side, we prove that 2-column Tetris (and 1-row Tetris) is polynomial. We also prove that the generalization of Tetris to larger k -omino pieces is NP-complete even when the board starts empty, and even when restricted to 3 columns or 2 rows or constant-size pieces. Finally, we present an animated Tetris font.

Keywords: complexity, hardness, video games, font

which piece sets are easy vs. hard? All previous Tetris NP-hardness proofs used five of the seven piece types. We also prove ASP-completeness of Tetris clearing, using three piece types, as well as versions of 3-Partition and Numerical 3-Dimensional Matching where all input integers are distinct. Finally, we prove NP-hardness of Tetris survival and clearing under the “hard drops only” and “20G” modes, using two piece types, improving on a previous “hard drops only” result that used five piece types.

In the pop
and must pack
filled squares;
by one row. A
number of ele
filled and clear
of pieces place
the first and
pieces, and the
Our results h
rotation, limit



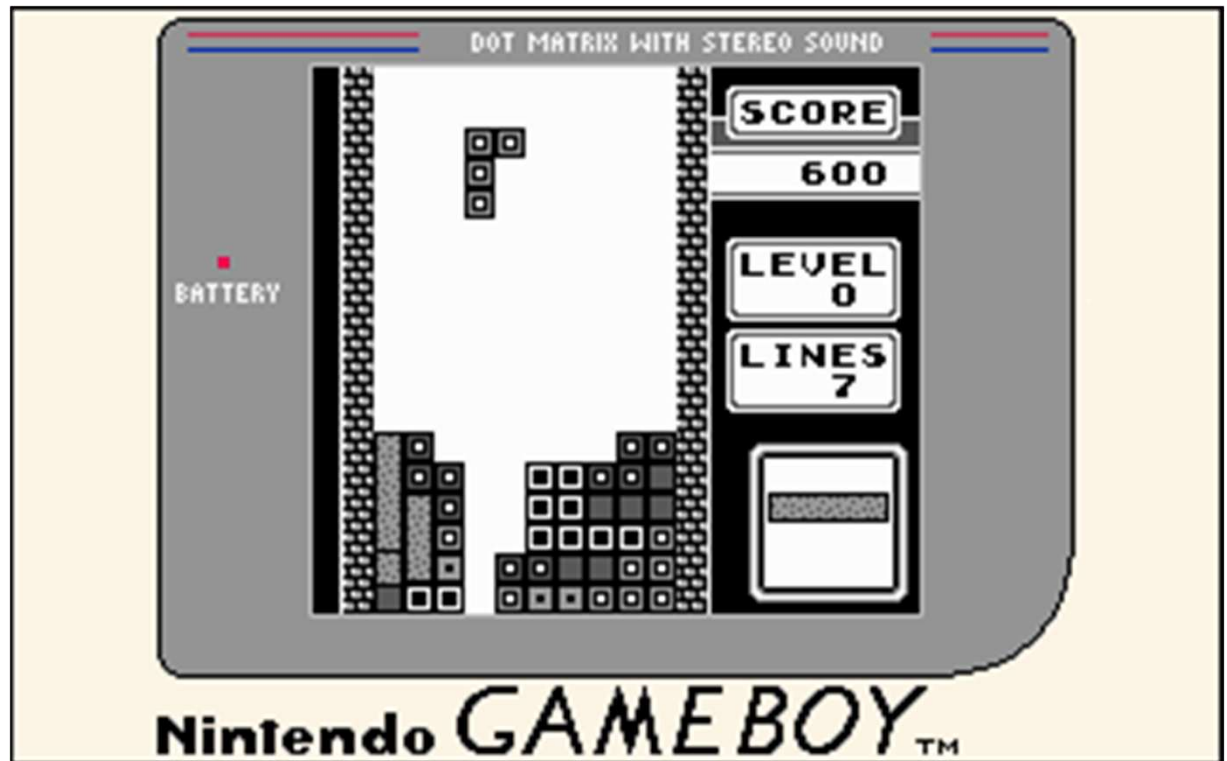
ICE

etris
ar

TEGLON?

- **T**etris for
- **G**ameboy
- **LO**gic
- **iN**version

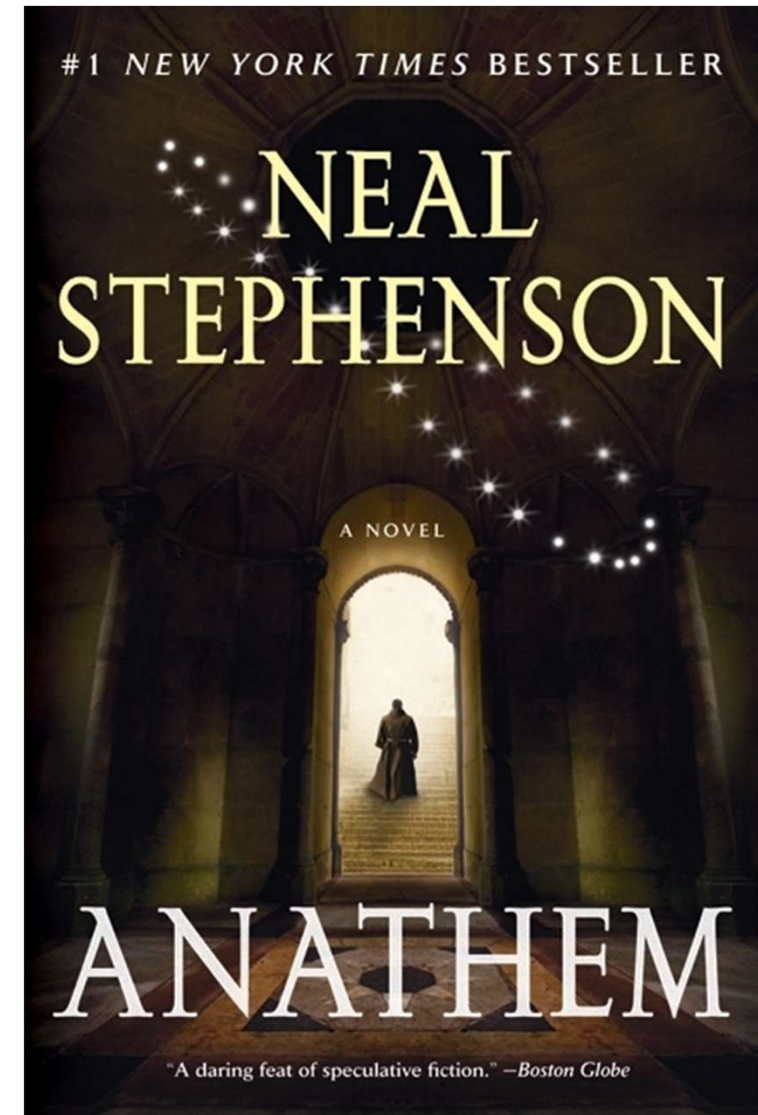
Where does the term originate from?



The **Teglon** is a complex problem of geometry and tiling which was originally developed at the [Temple of Orithena](#) on [Ecba](#). The problem is more than a theoretical exercise, an open space known as a Decagon is used to hold the physical tiles of the **Teglon** and immortalize a correct solution.

Seven varieties of **Teglon** tiles are used, each of which contains a groove or line on their top face. The objective is to create a pattern of tiles which provide an unbroken, continuous line over a large surface. The tiles are such that a solution is aperiodic and mathematically non-trivial.

To find a correct solution to the **Teglon** is extremely difficult. The aperiodic nature of the tiles mean that the solution requires a holistic awareness of the entire tiled surface. At [Elkhazg Fraa Jad](#) is able to find a correct solution and lay down the tiles in a single night, an unprecedented feat and only the fourth solution found at [Elkhazg](#). It is later implied that [Fraa Jad](#) is able to do this because of the unique abilities of [Thousands](#) to sense different cosmi.



Why come back to this topic?

<https://youtu.be/ScmhRNSrRP4>

+ 25

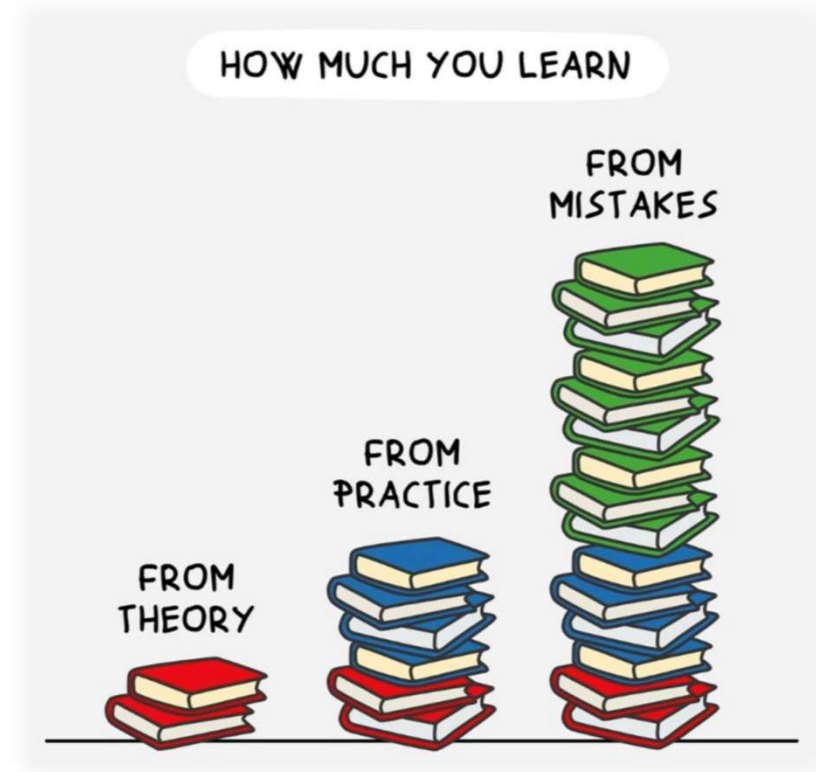
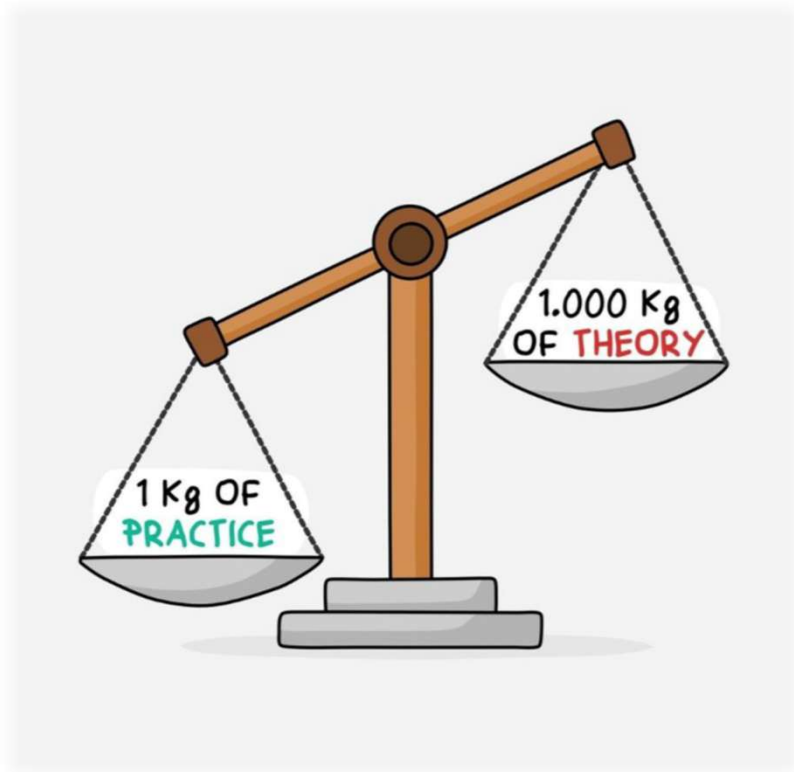
From C+ to C++

Modernizing a GameBoy Emulator

TOM TESCH



The problem we face as teachers...



The problem we face as teachers...

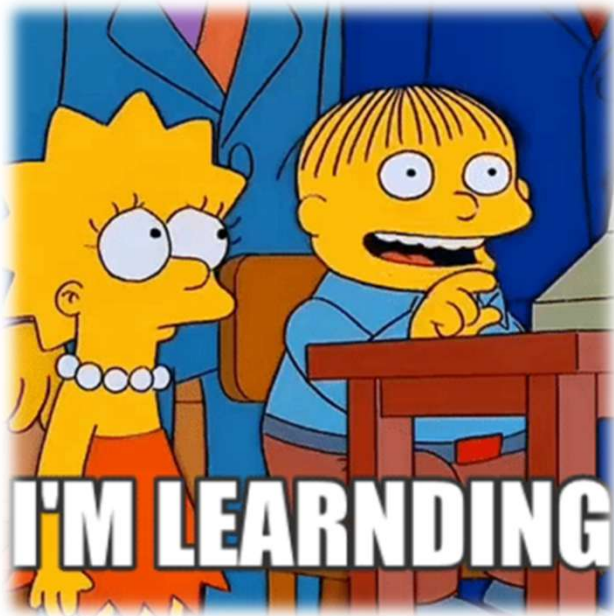
Slide & demo code we write:

Student code we see

The problem we face as teachers...

Slide & demo code we write:

Student code we see:



The problem we face as teachers...

Slide & demo code we write:



Student code we see:



From QT to olcPixelGameEngine

<https://github.com/OneLoneCoder/olcPixelGameEngine>

The official distribution of olcPixelGameEngine, a tool used in javidx9's YouTube videos and projects (<https://www.youtube.com/@javidx9>)

Single header include(!!!)

Compiles to windows, mac, linux and webassembly

Online playground

Can be run headless for test purposes





DEMO

New achievement: being called goat online...



James Lim ✓ • 1st

C++ | NUS CS

3w • 🌐



bro plays tetris live during a talk in CppCon25 🐐

i thoroughly enjoyed [Tom Tesch](#)'s refactor of a supposedly "C + std::vector" codebase to modern C++! also i am setting up a table

[#cpp](#)

[https://datacrystal.tcrf.net/wiki/Tetris_\(Game_Boy\)/RAM_map](https://datacrystal.tcrf.net/wiki/Tetris_(Game_Boy)/RAM_map)

Tetris (Game Boy)/RAM map

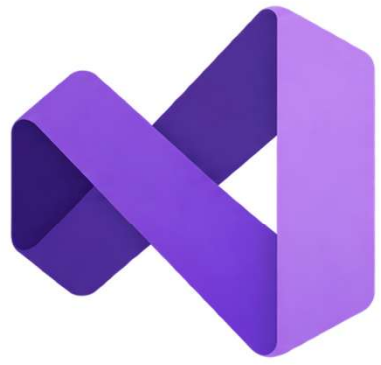
[< Tetris \(Game Boy\)](#)



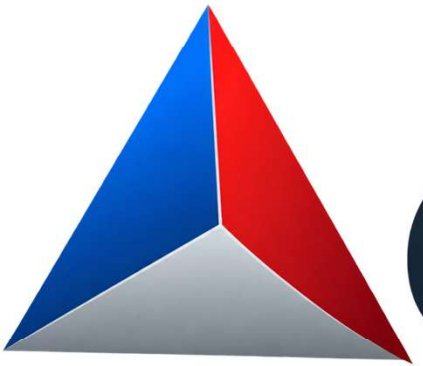
The following article is a [RAM map](#) for *Tetris (Game Boy)*.

WRAM

Address	Size	Description
-----	----	-----
C000		oam_buffer
C0A0		score_bcd
C0A3		ptrs_to_cleared_lines
C0AC		singles
C0B1		doubles
C0B6		triples
C0BB		tetrises



Visual Studio 26

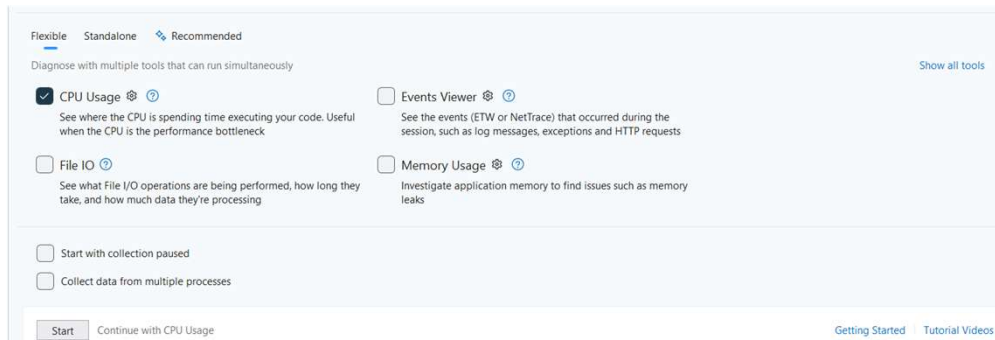


CMake



Visual Studio 26

- Excellent native C++ profiling
- first-class CMake support



Performance Profiling | Part 1 An Introduction
Microsoft Visual Studio
48K views • 6 years ago



Performance Profiling | Part 2: Choosing the right tool
Microsoft Visual Studio
23K views • 6 years ago

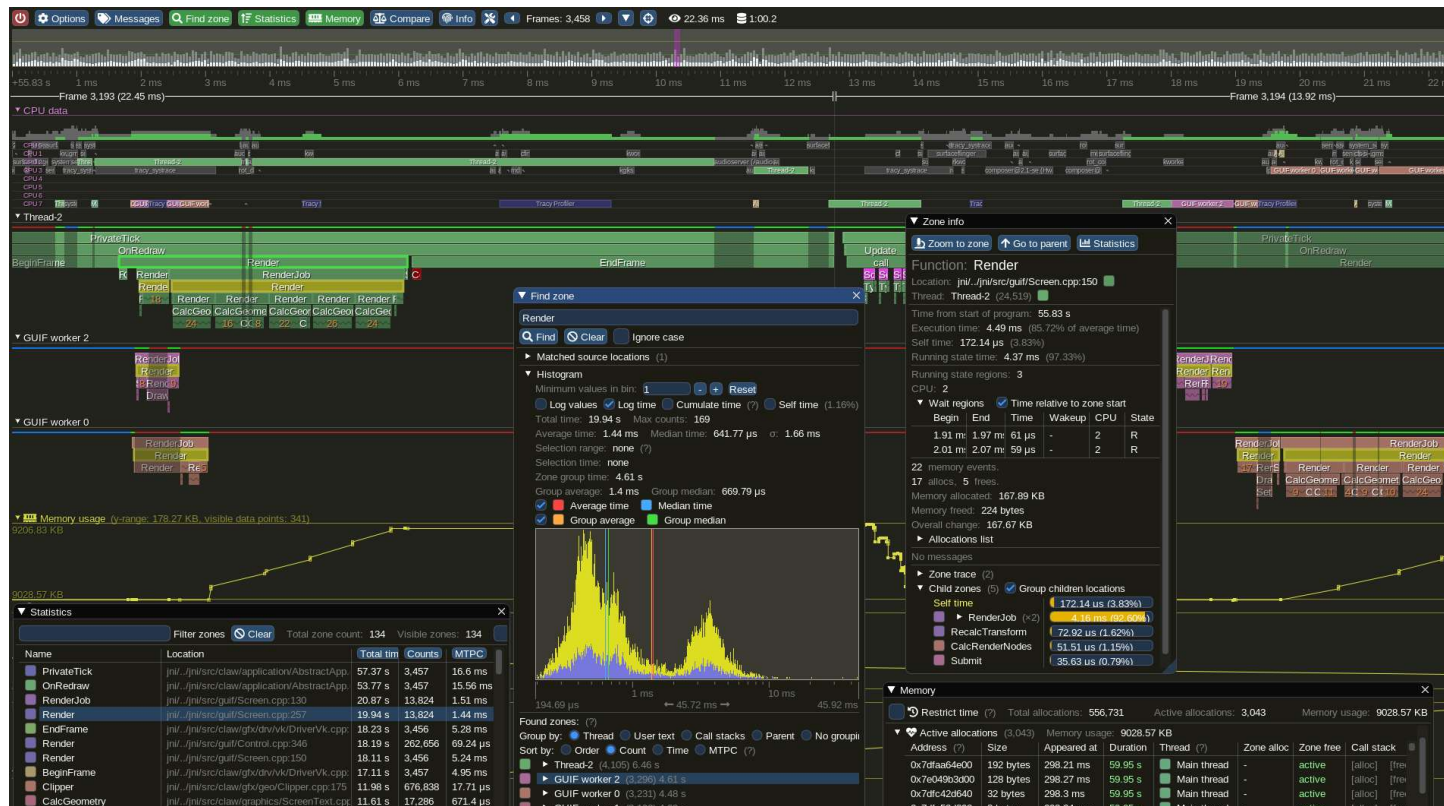


Performance with Profiling Part 3: Profiling and Production
Microsoft Visual Studio
10K views • 6 years ago

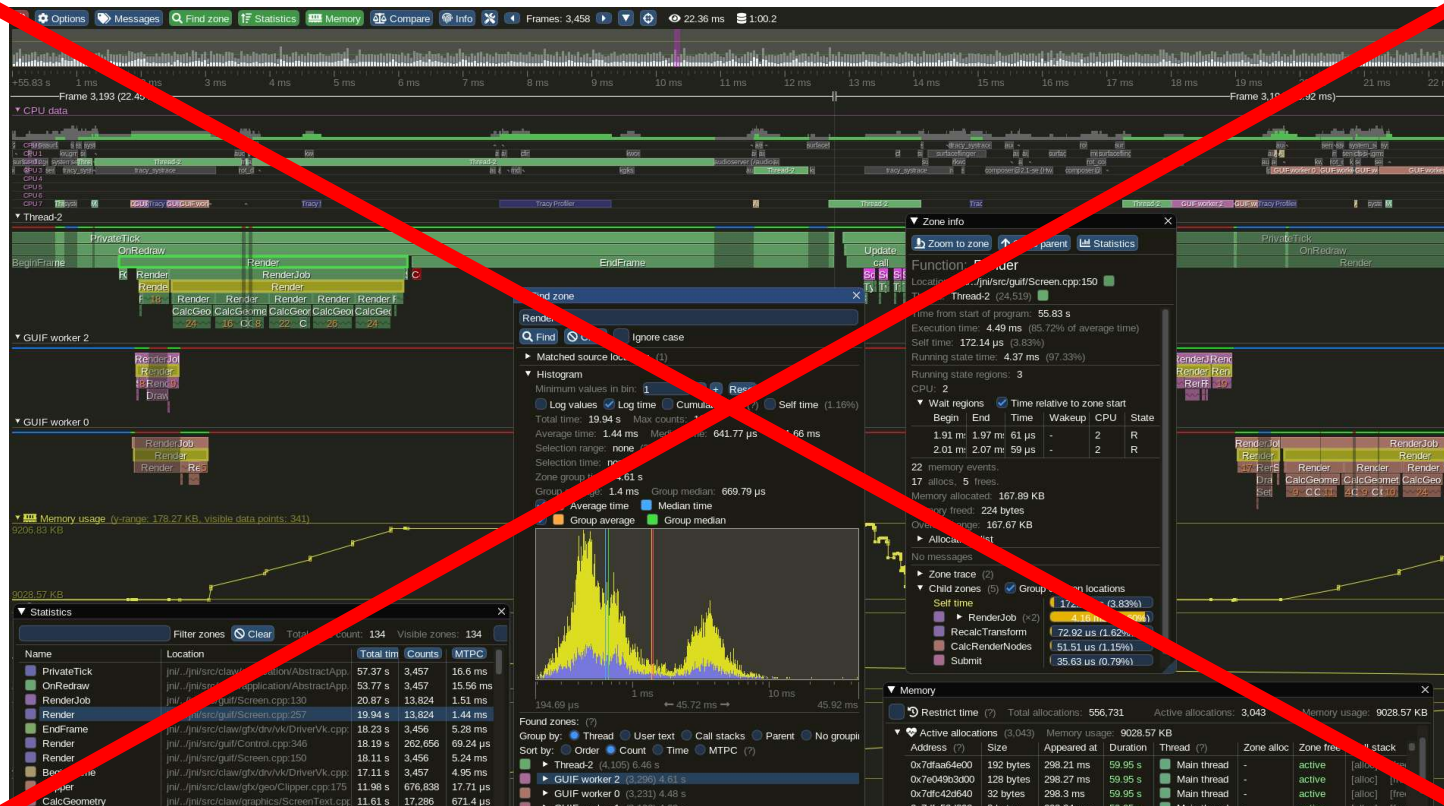


Performance Profiling | CPU Usage Tool
Microsoft Visual Studio
32K views • 5 years ago

<https://wolf.nereid.pl/projects/tracy-profiler/>



<https://wolf.nereid.pl/projects/tracy-profiler/>





Inside a Small Game Engine: An Architecture Tour in Modern C++

📅 Tuesday September 15, 2026 14:00 - 15:00 MDT

📍 Red Rock 6/7

According to VGInsights, about 10% of games released on Steam in 2024 used a custom engine, but those games represent 41% of units sold. Most "how an engine works" talks come from the AAA end of that distribution. This one comes from the other end of that distribution: a teaching-scale engine, where every system has to be small enough to understand from first principles. The problems are the same as in a big engine, but the smaller surface is what makes the architecture legible.

In this talk, we'll walk the spine of the engine in modern C++: allocators and handles in the core, the job system, the graphics abstraction question, the asset pipeline split between pipeline-time and runtime, and scene representation including ECS. We finish on serialization, where C++26 reflection (P2996) offers a path past today's reliance on macros and external code generators.

Presenters



Elias Farhan

Head of Department, SAE Institute Geneva

Elias Farhan is the head of the Games Programming department at SAE Institute Genève, as well as co-founder of indie game studio Team Kwakwa.

● [GameDev](#)

Subject [API Design](#), [Concurrency](#), [Performance](#), [Standard Library](#), [Tags](#) [Breakouts](#)





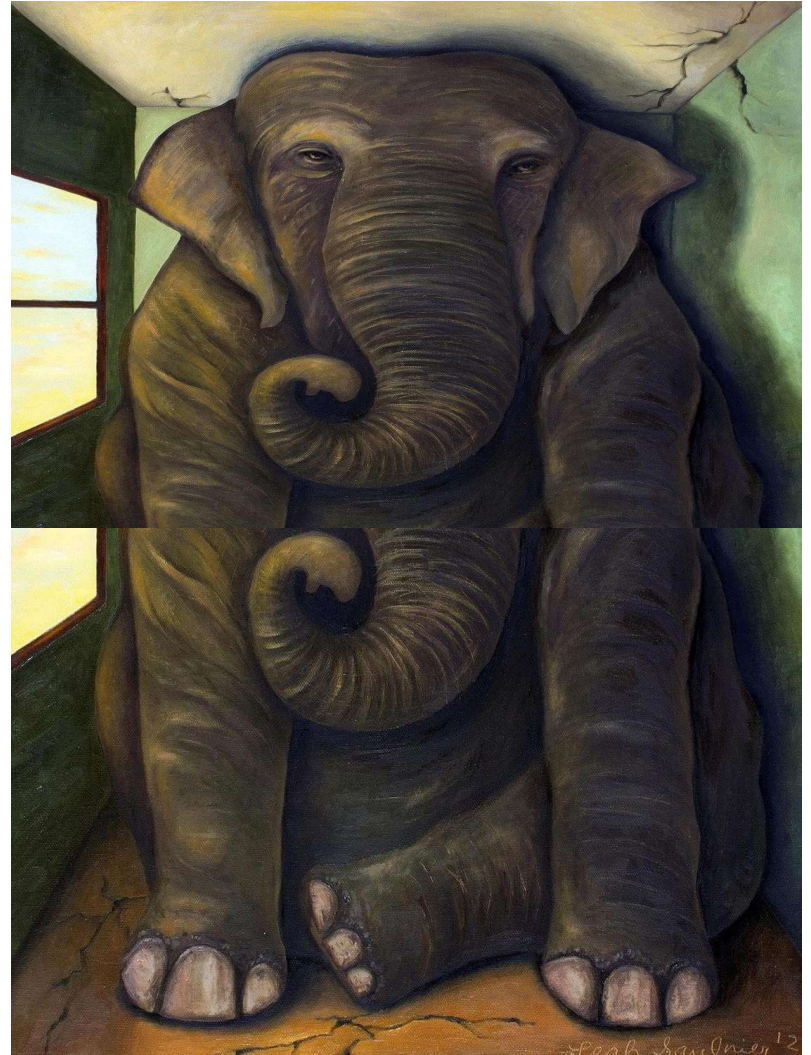


Codex



Codex use

- look things up
 - Graph making
 - Small repetitive tasks
 - Rubber ducking
 - Brainstorming
 - Git management
-
- Some assistance with slide layout



Codex use – what's the price of a fork

- look things up
- Graph making
- Small repetitive tasks
- Rubber ducking
- Brainstorming
- Git management

```
1  /*
2  * "THE BEER-WARE LICENSE" (Revision 42):
3  * Tom Tesch's migration code (with Codex assistance). As long as you retain
4  * this notice you can do whatever you want with this stuff. If we meet some
5  * day, and you think this stuff is worth it, you can buy me a beer in return.
6  * Tom
7  */
8  /*
9  * -----
10 * "THE BEER-WARE LICENSE" (Revision 42):
11 * Olivier Delbeke wrote this file. As long as you retain this notice you
12 * can do whatever you want with this stuff. If we meet some day, and you think
13 * this stuff is worth it, you can buy me a beer in return. Olivier
14 * -----
15 */
```

- Some assistance with slide layout

Code attribution

Original emulator

Olivier Delbeke

Migration and subsequent changes

Tom Tesch

Beer-Ware (Revision 42) • full notices preserved in speaker notes

Measuring an honest baseline

Measuring an **honest** baseline

Last year

HWiNFO® 64 v8.52-6060 @ HP OMEN Laptop 15-en1xxx - System Summary

CPU

 **AMD Ryzen 7 5800H** 7 nm

Stepping: **CZN-A0** TDP: **45 W**

Codename: **Cezanne** MCU: **A50000C**

OPN: **100-00000295**

CPU #0 Platform: **FP6**

Core: **8 / 16** Cache L1: **8x32 + 8x32** L2: **8x512** L3: **16M**

Features

MMX	3DNow!	3DNow!-2	SSE	SSE-2	SSE-3	SSSE-3
SSE4A	SSE4.1	SSE4.2	AVX	AVX2	AVX-512	AVX10
BMI2	ABM	TBM	FMA	ADX	XOP	AMX
DEP	AMD-V	SVMX	SMEP	SMAP	TSX	MPX
EM64T	EIST	TM1	TM2	HTT	CPB	SST
AES-NI	RDRAND	RDSEED	SHA	SGX	SME	APX

Operating Point	Clock	Ratio	Bus	VID
Minimum Clock	400.0 MHz	x4.00	100.0 MHz	-
Base Clock	3200.0 MHz	x32.00	100.0 MHz	-
Boost Max	4450.0 MHz	x44.50	100.0 MHz	-
PBO Max	4450.0 MHz	x44.50	100.0 MHz	-
Avg. Active Clock	3739.5 MHz	x37.46	99.8 MHz	1.1609 V
Avg. Effective Clock	1412.6 MHz	x14.15	-	-

Motherboard

HP 88D1

Chipset: **AMD Promontory/Bixby FCH**

BIOS Date: **03/16/2022** Version: **F.13** **UEFI**

Memory

Size: **16 GB** Type: **DDR4 SDRAM**

Clock: **1597.1 MHz** = **16.00** x **99.8 MHz**

Mode: **Dual-Channel** CR: **1T**

Timing: **22 - 22 - 22 - 52** tRC: **74** tRFC: **560**

Memory Modules

#0 [P0 CHANNEL A/Bottom - Slot 1 (left)]: **Samsung M471A1K43**

Size: **8 GB** Clock: **1600 MHz** ECC: **No**

Type: **DDR4-3200 / PC4-25600 DDR4 SDRAM SO-DIMM**

Clock	tCL	tRCD	tRP	tRAS	RC	Ext.	V
1600	22	22	22	52	74	-	1.20
1467	21	21	21	47	68	-	1.20
1333	19	19	19	43	61	-	1.20
1200	17	17	17	39	55	-	1.20
1067	15	15	15	35	49	-	1.20
933.3	13	13	13	30	43	-	1.20
800.0	11	11	11	26	37	-	1.20
666.7	10	10	10	22	31	-	1.20

GPU

 **NVIDIA GeForce RTX 3060 Laptop [HP]**

NVIDIA GeForce RTX 3060 Laptop

GA106M

PCIe v4.0 x16 (16.0 GT/s) @ x8 (2.5 GT/s)

GPU #0 6 GB **GDDR6 SDRAM** 192-bit

ROPs / TMUs: **48 / 120** SH/RT/TC: **3840 / 30 / 120**

Current Clocks (MHz)

GPU: **210.0** Memory: **101.2** Video: **555.0**

Operating System **UEFI Boot** **Secure Boot** **TPM** **HVCI**

Microsoft Windows 11 Enterprise (x64) Build 26200.8655 (25H2)

Drives

Interface	Model [Capacity]
✓ NVMe x4 8.0 GT/s	SAMSUNG MZVLB512HBJQ-000H1 [512 GB]

This year

HWiNFO® 64 v8.52-6060 @ LENOVO LOQ 15AHP10 - System Summary

CPU



AMD Ryzen 7 250 4 nm

Stepping	HPT1-A2	TDP	28 W
Codename	Hawk Point 1	MCU	A70520A
SSPEC	-		

CPU #0 ▾ Platform FP8

Core	8 / 16	Cache L1	8x32 + 8x32	L2	8x1M	L3	16M
	-		-		-		-

Features

MMX	3DNow!	3DNow!-2	SSE	SSE-2	SSE-3	SSSE-3
SSE4A	SSE4.1	SSE4.2	AVX	AVX2	AVX-512	AVX10
BMI2	ABM	TBM	FMA	ADX	XOP	AMX
DEP	AMD-V	SMX	SMEP	SMAP	TSX	MPX
EM64T	EIST	TM1	TM2	HTT	CPB	SST
AES-NI	RDRAND	RDSEED	SHA	SGX	SME	APX

Operating Point	Clock	Ratio	Bus	VID
Minimum Clock	400.0 MHz	x4.00	100.0 MHz	-
Base Clock	3300.0 MHz	x33.00	100.0 MHz	-
Boost Max	5100.0 MHz	x51.00	100.0 MHz	-
Avg. Active Clock	3352.9 MHz	x33.59	99.8 MHz	1.2988 V
Avg. Effective Clock	186.4 MHz	x1.87	-	-

Motherboard

LENOVO LNVN8161216

Chipset AMD Promontory/Bixby FCH

BIOS Date 06/20/2025 Version R8CN23WW UEFI

Memory

Size	32 GB	Type	DDR5 SDRAM
Clock	1175.3 MHz	=	11.77 x 99.8 MHz
Mode	Dual-Channel	CR	1T
Timing	46 - 45 - 45 - 90	tRC	135 tRFC 447

Memory Modules

#0 [P0 CHANNEL A/DIMM 0]: Samsung M425R2GA3EB0-CWMOD ▾

Size	16 GB	Clock	2800 MHz	ECC	No
Type	DDR5 SDRAM				

Clock	tCL	tRCD	tRP	tRAS	RC	Ext.	V
-------	-----	------	-----	------	----	------	---

GPU



NVIDIA GeForce RTX 5060 Laptop (GB206M) [Lenov
NVIDIA GeForce RTX 5060 Laptop

GB206M

PCIe v5.0 x16 (32.0 GT/s) @ x8 (2.5 GT/s)

GPU #0 ▾ 7.96 GB GDDR7 SDRAM 128-bit

ROPs 48 SH/RT/TC 3328 / 26 / 104

Current Clocks (MHz)

GPU	180.0	Memory	50.6	Video	600.0
-----	-------	--------	------	-------	-------

Operating System UEFI Boot Secure Boot TPM HVCI

Microsoft Windows 11 Enterprise (x64) Build 26200.9445 (25H2)

Drives

Interface	Model [Capacity]
✔ NVMe x4 16.0 GT/s	Micron MTFDKCD1T0QGN-1BN1AABLA...

New laptop

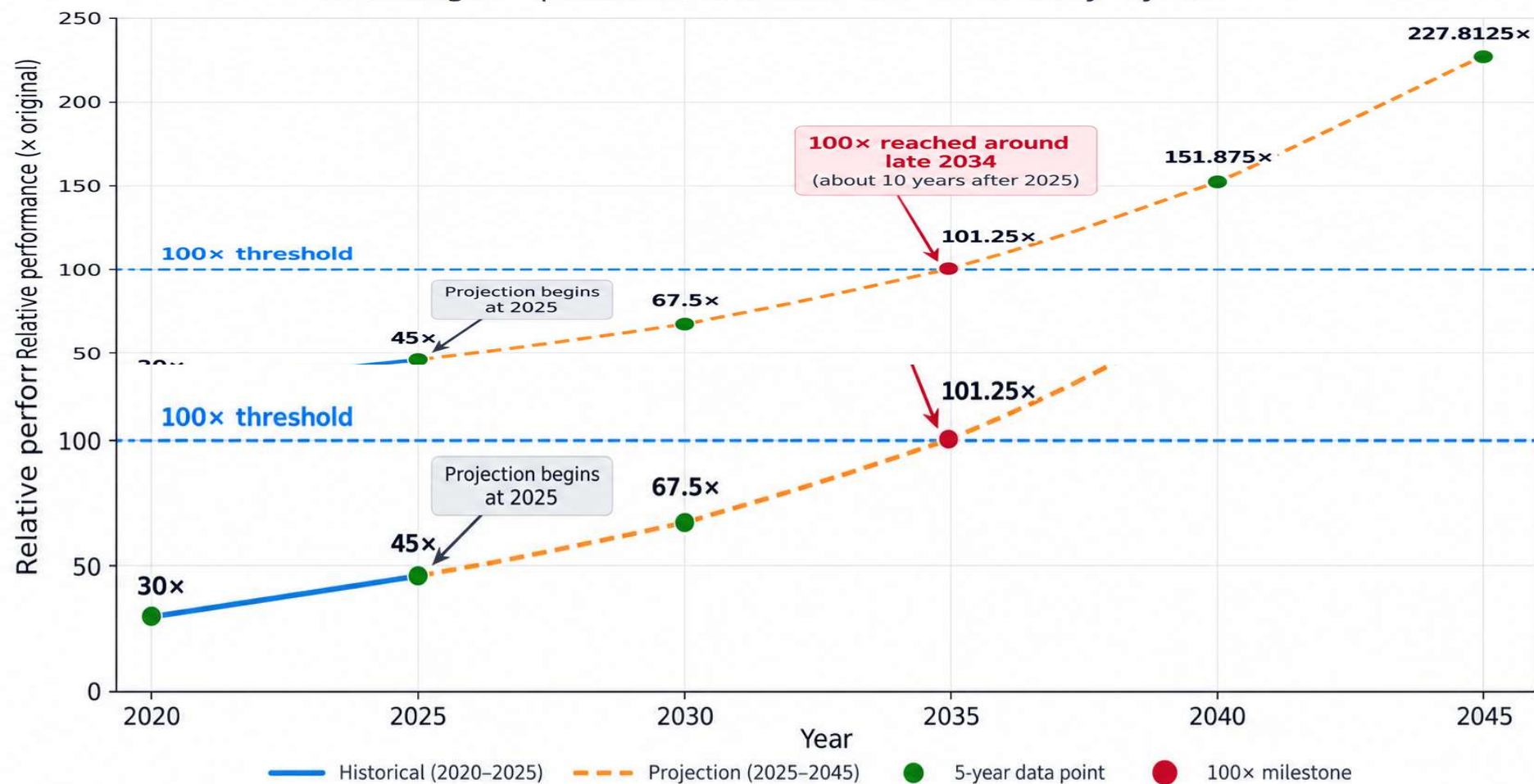
	HP OMEN 15	Lenovo LOQ 15AHP10
CPU	Ryzen 7 5800H	Ryzen 7 250
Cores / threads	8 / 16	8 / 16
Max boost	4.45 GHz	5.10 GHz
GPU	RTX 3060 6 GB	RTX 5060 8 GB
RAM	16 GB DDR4-3200	32 GB DDR5-5600
SSD	512 GB PCIe 3.0 NVMe	1 TB PCIe 4.0 NVMe

New laptop

	HP OMEN 15	Lenovo LOQ 15AHP10
CPU	Ryzen 7 5800H	Ryzen 7 250
Cores / threads	8 / 16	8 / 16
Max boost	4.45 GHz	5.10 GHz
GPU	RTX 3060 6 GB	RTX 5060 8 GB
RAM	16 GB DDR4-3200	32 GB DDR5-5600
SSD	512 GB PCIe 3.0 NVMe	1 TB PCIe 4.0 NVMe
Game Boy benchmark	31 FPS	47 FPS (+52%)

Projected emulator performance growth

Assuming 30× performance in 2020 and +50% every 5 years



Questions?

14:45
MDT



Back to Basics: C++20 Concepts

Amir Kirsh



The Value of Idiomatic Code

John Pavan



Escaping 1996: Meta-Modern C++ Techniques for Embedded Systems

Michael Caisse



In Pursuit of a 6,000 FPS Game Boy Emulator

Tom Tesch

What 25 Days of Optimisations Taught Me About Compilers

Matt Godbolt

Concurrency for Modern CPUs - Lock-Free or Lock-based?

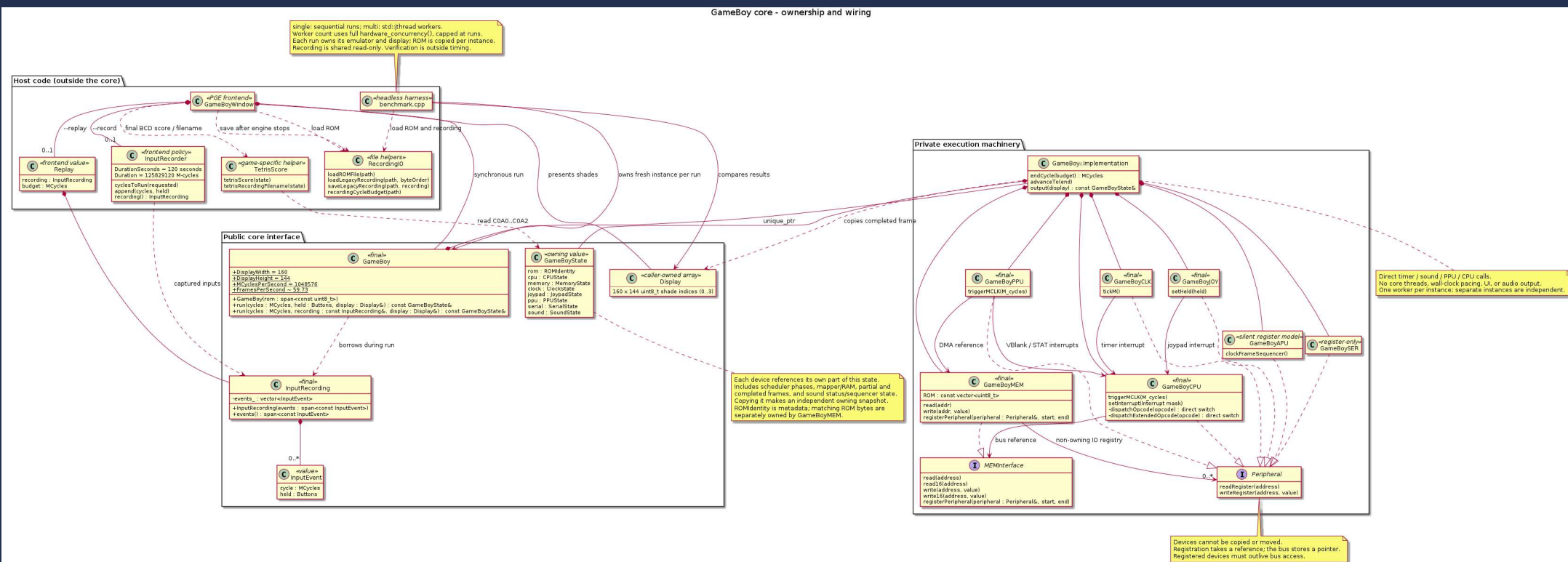
Fedor Pikus



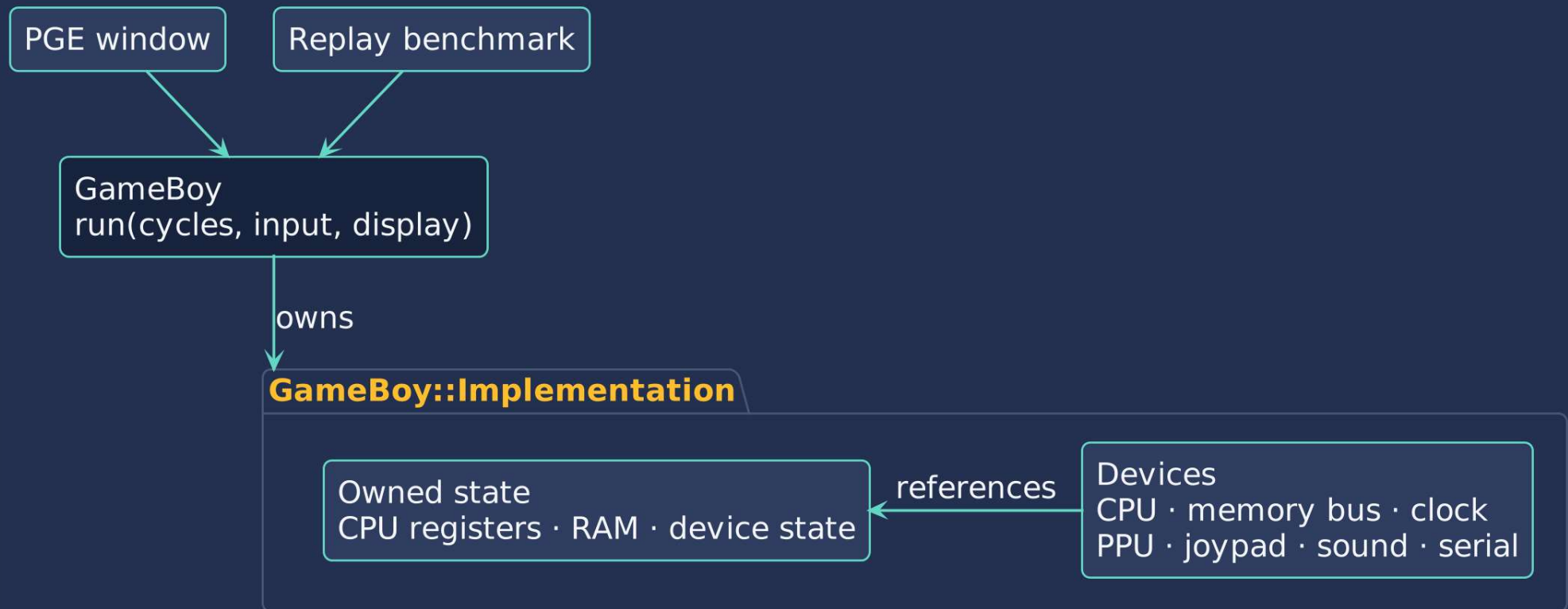
Some ground rules

- Must run in WSL and on Windows
 - GCC and MSVC
 - Results must be stable
 - CMAKE / VS26
-
- This is a recreation of an otherwise very messy process

Full ownership diagram



How the emulator is put together



One owner; devices refer to its state

```
struct GameBoy::Implementation {  
    GameBoyState state;  
    GameBoyMEM MEM;  
    GameBoyCPU CPU;  
    GameBoyCLK CLK;  
    // Other devices follow.  
  
    explicit Implementation(std::span<std::uint8_t const> rom)  
        : MEM{rom, state.memory, state.rom},  
          CPU{MEM, state.cpu},  
          CLK{MEM, CPU, state.clock} /* ... */ {}  
};
```

A run stays on the caller's thread

```
GameBoyState const& GameBoy::run(  
    MCycles cycles, Buttons held, Display& display) {  
    auto& core{*implementation};  
    auto const end{core.endCycle(cycles)};  
  
    core.JOY.setHeld(held);  
    core.advanceTo(end);  
    return core.output(display);  
}
```

The same two minutes, every time

Emulated duration

120 s

125,829,120 M-cycles

Completed frames

7,148

Same recorded button input

Same ROM, inputs and cycle budget within each comparison.

What the timer includes

```
auto const begin{std::chrono::steady_clock::now()};  
  
experiment.state =  
    &experiment.gameboy->run(  
        budget, recording, experiment.display);  
  
auto const end{std::chrono::steady_clock::now()};  
experiment.elapsed = end - begin;
```

Load and construct before. Compare and print after.

Check the state, not just the screenshot

```
if (state != expectedState ||  
    display != expectedDisplay)  
    throw std::runtime_error{  
        "Repeated execution produced "  
        "different state/frame data"};
```

Repeat the run. Then compare old and new implementations.

Five changes

Change	What we changed
Modernization	Interfaces and opcode bindings
LTO	Compiler visibility across files
M-cycle batching	Clock and sound update frequency
Switch dispatch	The opcode-to-handler mapping
Tile-row rendering	Reuse of row data between pixels

Lambdas, still inside std::function

```
instructions[0x00] = [this] { Instr_NOP(); };  
instructions[0x01] = [this] { Instr_LD_R16_D16(BC); };  
instructions[0x02] = [this] { Instr_LD_MR16_A(BC); };  
  
instructions[opcode]();
```

We had changed the callable. We had kept the wrapper.

The first change slowed Windows down

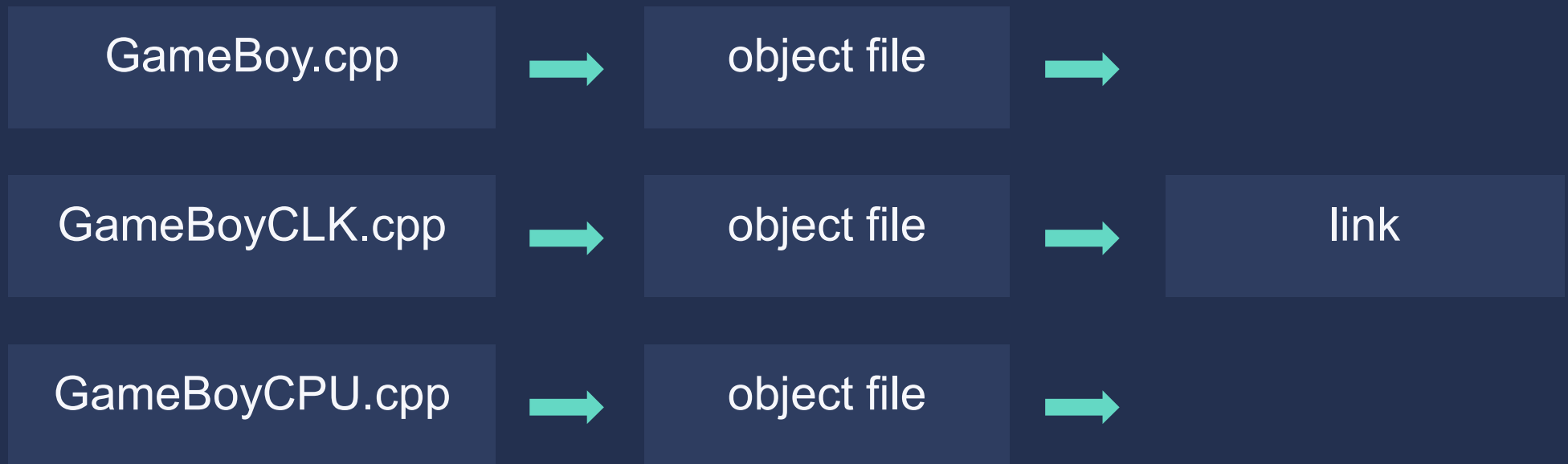
	Before FPS	After FPS	Change
Windows	2,828.1	2,640.1	-6.6%
WSL	2,663.3	2,731.0	+2.5%

44.2×

45.7×

Several changes were merged; this does not isolate the lambdas.

The compiler cannot always see the body



LTO carries compiler information through to the final link.

Inlining can remove more than a call

```
// Defined in another source file:  
unsigned advance(unsigned cycle) {  
    return cycle + 4;  
}  
  
auto const next = advance(cycle);  
if (next == cycle)  
    handleNoProgress();
```

With the body visible, this branch is impossible.

Enable LTO for the libraries and executable

```
set(CMAKE_INTERPROCEDURAL_OPTIMIZATION_RELEASE  
    ${GAMEBOY_ENABLE_LTO})  
  
set(CMAKE_INTERPROCEDURAL_OPTIMIZATION_DEBUG OFF)
```

MSVC: /GL at compile time, /LTCG at link time

GCC: -O2 -flto

More optimization at link time

	Before FPS	After FPS	Change
Windows	2,640.1	3,511.3	+33.0%
WSL	2,731.0	4,739.8	+73.6%

58.8×

79.4×

Tradeoff: more build work, and toolchain-dependent gains.

The profiler needs careful interpretation

CPU.triggerMCLK disappeared from the call tree.

It had been inlined into advanceTo().

Time in read() also includes address decoding.

Source-line attribution is not a breakdown of hardware stalls.

Four T-cycles make one M-cycle



1 M-cycle

503,316,480 T-cycles in this replay.

Before: four clock and sound updates

```
while (state.clock.M_cycles < end) {  
    for (unsigned tick{}; tick < 4; ++tick) {  
        CLK.tick();  
        APU.triggerTCLK(state.clock.T_cycles);  
    }  
  
    PPU.triggerMCLK(state.clock.M_cycles);  
    CPU.triggerMCLK(state.clock.M_cycles);  
}
```

Advance the counter by four

Before

```
++state.T_cycles;  
if ((state.T_cycles & 0xFF)  
    == 0)  
    ++state.DIV;  
  
if ((state.T_cycles & 3) != 0)  
    return;  
  
++state.M_cycles;
```

After

```
state.T_cycles +=  
    TCyclesPerMCycle;  
  
if ((state.T_cycles &  
    (DividerPeriodTCycles - 1))  
    == 0)  
    ++state.DIV;  
++state.M_cycles;
```


Most sound checks did no work

Old sound entry-point invocations

503,316,480

61,440

possible sequencer boundaries

One boundary per 8,192 T-cycles. No audio samples generated.

After: update only at the right boundary

```
while (state.clock.M_cycles < end) {  
    CLK.tickM();  
    if ((state.clock.T_cycles &  
        (SoundSequencerPeriodTCycles - 1)) == 0)  
        APU.clockFrameSequencer();  
  
    PPU.triggerMCLK(state.clock.M_cycles);  
    CPU.triggerMCLK(state.clock.M_cycles);  
}
```

The order stays: clock → sound → PPU → CPU.

The same condition, written with %

```
while (state.clock.M_cycles < end) {  
    CLK.tickM();  
    if ((state.clock.T_cycles %  
        SoundSequencerPeriodTCycles) == 0)  
        APU.clockFrameSequencer();  
  
    PPU.triggerMCLK(state.clock.M_cycles);  
    CPU.triggerMCLK(state.clock.M_cycles);  
}
```

The order stays: clock → sound → PPU → CPU.

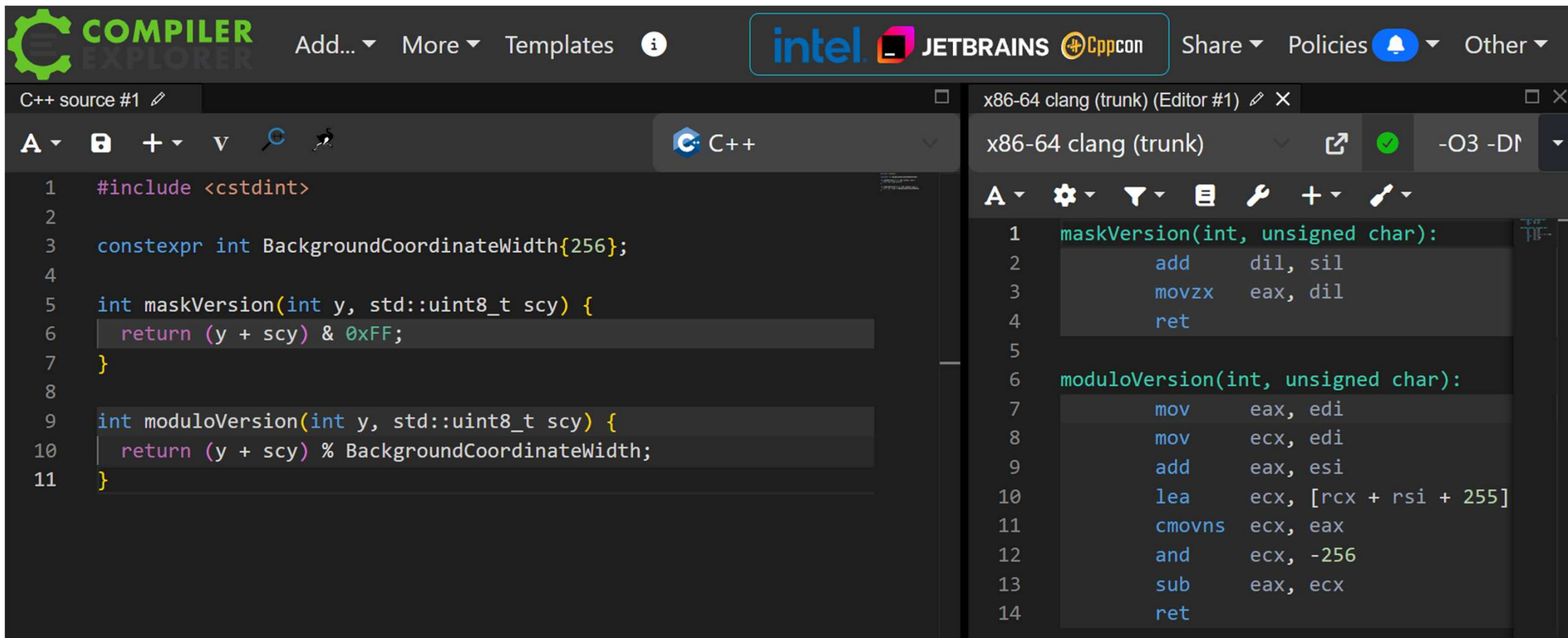
I asked Codex to change all similar cases to modulo for readability

Aside

4% loss of performance!

<https://godbolt.org/z/r43P877o6>

Aside



The screenshot displays the Godbolt compiler explorer interface. The left pane shows the C++ source code for a program. The right pane shows the assembly output generated by the x86-64 clang (trunk) compiler.

C++ source code (C++ source #1):

```
1 #include <cstdint>
2
3 constexpr int BackgroundCoordinateWidth{256};
4
5 int maskVersion(int y, std::uint8_t scy) {
6     return (y + scy) & 0xFF;
7 }
8
9 int moduloVersion(int y, std::uint8_t scy) {
10    return (y + scy) % BackgroundCoordinateWidth;
11 }
```

Assembly output (x86-64 clang (trunk) (Editor #1)):

```
1 maskVersion(int, unsigned char):
2     add     dil, sil
3     movzx   eax, dil
4     ret
5
6 moduloVersion(int, unsigned char):
7     mov     eax, edi
8     mov     ecx, edi
9     add     eax, esi
10    lea     ecx, [rcx + rsi + 255]
11    cmovns  ecx, eax
12    and     ecx, -256
13    sub     eax, ecx
14    ret
```

We can all read this right?

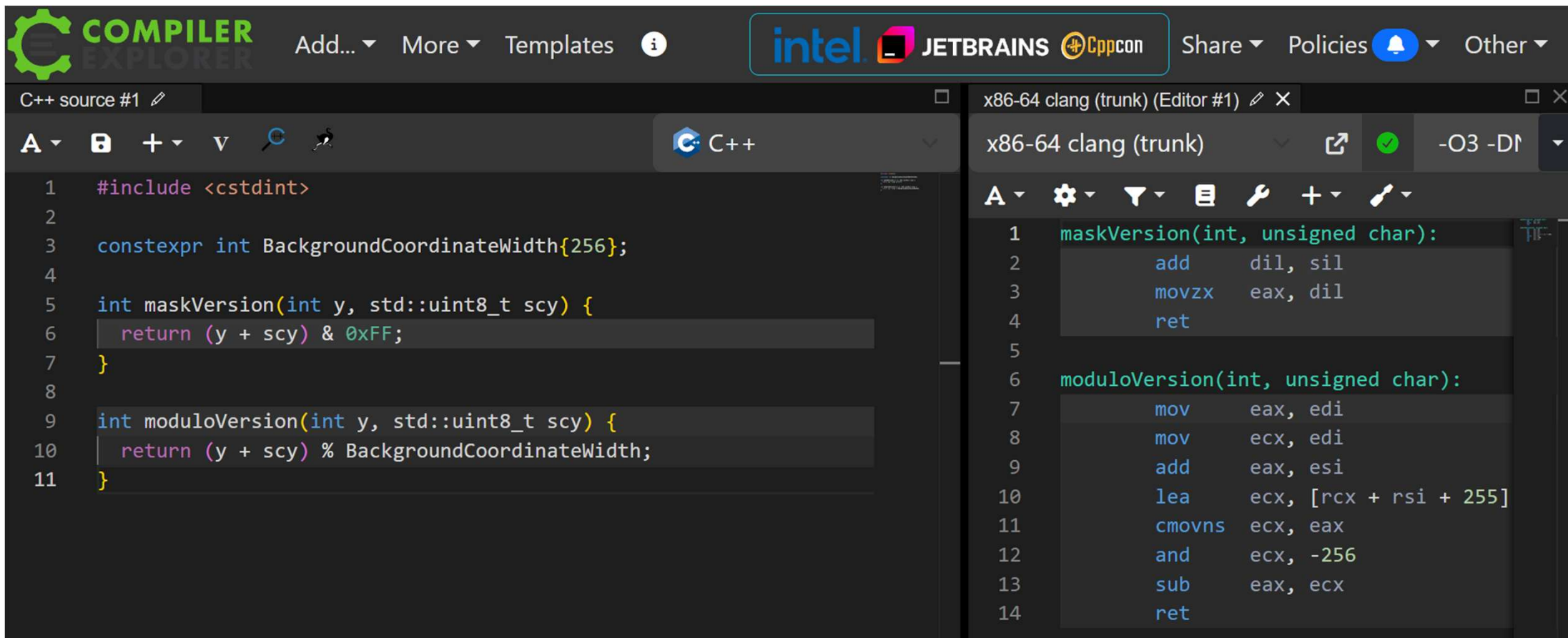
```
add    esi, edi  
movzx  eax, sil  
ret
```

We can all read this right?

```
add    esi, edi    ; y += scy
movzx  eax, sil     ; keep bits 0-7, zero-extend to int
ret     ; return eax
```


<https://godbolt.org/z/r43P877o6>

Aside



The screenshot displays the Godbolt compiler explorer interface. The left pane shows the C++ source code for a program. The right pane shows the assembly output generated by the x86-64 clang (trunk) compiler.

C++ source code (C++ source #1):

```
1 #include <cstdint>
2
3 constexpr int BackgroundCoordinateWidth{256};
4
5 int maskVersion(int y, std::uint8_t scy) {
6     return (y + scy) & 0xFF;
7 }
8
9 int moduloVersion(int y, std::uint8_t scy) {
10    return (y + scy) % BackgroundCoordinateWidth;
11 }
```

Assembly output (x86-64 clang (trunk) (Editor #1)):

```
1 maskVersion(int, unsigned char):
2     add    dil, sil
3     movzx  eax, dil
4     ret
5
6 moduloVersion(int, unsigned char):
7     mov    eax, edi
8     mov    ecx, edi
9     add    eax, esi
10    lea     ecx, [rcx + rsi + 255]
11    cmovns ecx, eax
12    and     ecx, -256
13    sub     eax, ecx
14    ret
```

The unsigned int is causing the trouble

```
if (x >= 0)
    return x & 0xFF;
else
    return -((-x) & 0xFF);
```

```
movzx esi, sil
add    esi, edi
```

; Determine whether x is negative

```
mov    edx, esi
sar    edx, 31
shr    edx, 24
```

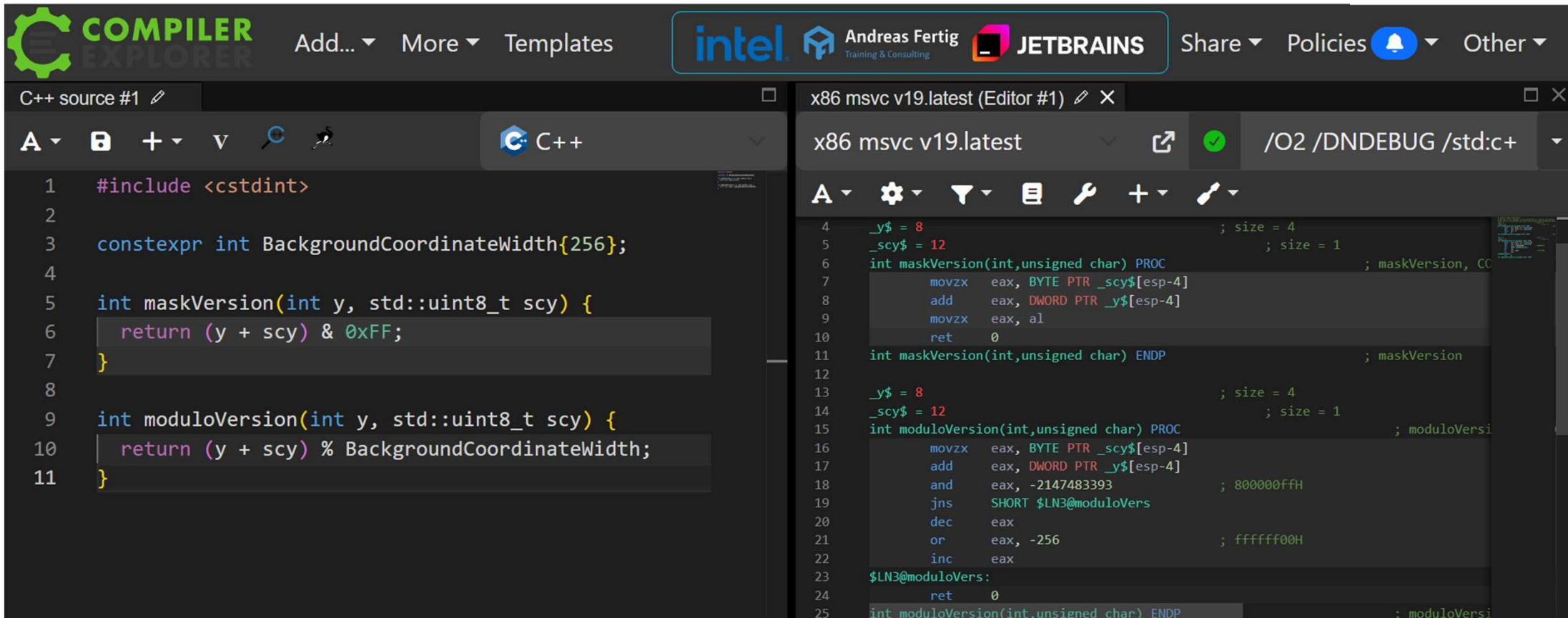
; Correct for C++ signed-remainder semantics

```
lea    eax, [rsi+edx]
movzx  eax, al
sub    eax, edx
```

```
ret
```

<https://godbolt.org/z/6Ye8zv6hP>

Aside



The screenshot displays the Godbolt compiler explorer interface. The left pane shows the C++ source code for a program. The right pane shows the x86 assembly output generated by the compiler (msvc v19.latest).

C++ source code (C++ source #1):

```
1  #include <cstdint>
2
3  constexpr int BackgroundCoordinateWidth{256};
4
5  int maskVersion(int y, std::uint8_t scy) {
6      return (y + scy) & 0xFF;
7  }
8
9  int moduloVersion(int y, std::uint8_t scy) {
10     return (y + scy) % BackgroundCoordinateWidth;
11 }
```

Assembly output (x86 msvc v19.latest):

```
4  _y$ = 8 ; size = 4
5  _scy$ = 12 ; size = 1
6  int maskVersion(int,unsigned char) PROC ; maskVersion, CC
7      movzx    eax, BYTE PTR _scy$[esp-4]
8      add     eax, DWORD PTR _y$[esp-4]
9      movzx    eax, al
10     ret     0
11 int maskVersion(int,unsigned char) ENDP ; maskVersion
12
13 _y$ = 8 ; size = 4
14 _scy$ = 12 ; size = 1
15 int moduloVersion(int,unsigned char) PROC ; moduloVersi
16     movzx    eax, BYTE PTR _scy$[esp-4]
17     add     eax, DWORD PTR _y$[esp-4]
18     and     eax, -2147483393 ; 800000ffH
19     jns     SHORT $LN3@moduloVers
20     dec     eax
21     or      eax, -256 ; ffffffff00H
22     inc     eax
23 $LN3@moduloVers:
24     ret     0
25 int moduloVersion(int,unsigned char) ENDP ; moduloVersi
```

Batching was the biggest Windows gain

	Before FPS	After FPS	Change
Windows	3,511.3	5,302.5	+51.0%
WSL	4,739.8	6,572.3	+38.7%

88.8×

110.0×

Condition: every relevant boundary must fit the four-T-cycle step.

Fetching and dispatching an instruction

```
if (elapsedMCycles >= nextInstructionCycle) {  
    if (!halted) {  
        std::uint8_t const opcode{MEM.read(PC++)};  
        dispatchOpcode(opcode);  
    }  
  
    // Existing interrupt handling follows.  
}
```

The same handlers, called directly

Before

```
instructions[0x00] =  
    [this] { Instr_NOP(); };  
  
instructions[0x01] = [this] {  
    Instr_LD_R16_D16(BC);  
};  
  
instructions[opcode]();
```

After

```
switch (opcode) {  
    case 0x00:  
        Instr_NOP();  
        return;  
    case 0x01:  
        Instr_LD_R16_D16(BC);  
        return;  
}
```

The register is known at compile time

```
void GameBoyCPU::Instr_LD_R16_D16(std::uint16_t& r16) {  
    r16 = MEM.read16(PC);  
    PC += 2;  
    nextInstructionCycle += 3;  
}
```

```
BC = MEM.read16(PC);  
PC += 2;  
nextInstructionCycle += 3;
```

Bottom: the possible effect of inlining the 0x01 case.

The generated code got smaller

GCC text section before

340,968 B

After the switch change

152,760 B

Jump table. Some handlers inlined. Others still called.

We did not measure cache misses or branch misses.

Direct dispatch helped both builds

	Before FPS	After FPS	Change
Windows	5,302.5	5,903.5	+11.3%
WSL	6,572.3	6,717.6	+2.2%

98.8×

112.5×

Cost: 512 explicit opcode mappings to keep correct.

How the Game Boy builds a background



SCX / SCY scroll the background.

WX / WY position the window layer.

Two bytes encode eight pixels

Low bits	0	1	0	1	0	1	0	1
High bits	0	0	1	1	0	0	1	1
Color index	0	1	2	3	0	1	2	3

```
colorIndex = ((lowByte >> bit) & 1)
             | (((highByte >> bit) & 1) << 1);
```

We fetched the same row for every pixel

```
for (int x = firstPixel; x <= lastPixel; ++x) {  
    auto source = selectBackgroundOrWindow(x, y);  
    auto row = getTileRow(source);  
    auto colorIndex = decodePixel(row, source.x);  
  
    backgroundColors[x] = colorIndex;  
    putPixel(x, y, paletteShade(colorIndex));  
}
```

Simplified before-code. Most neighbors use the same row bytes.

Choose the layer and load one row

```
bool const inWindow{windowEnabled && x >= windowStart};  
int const sourceX{  
    inWindow ? x - windowStart : (x + SCX) & 0xFF};  
  
auto const row{getTileRow(  
    inWindow ? windowTileMap : backgroundTileMap,  
    usesSignedTileIndices, sourceX,  
    inWindow ? windowY : backgroundY)};
```

Stop when the row data could change

```
int const pixelInTile{sourceX & (TileWidth - 1)};

int groupEnd{
    std::min(lastPixel + 1, x + TileWidth - pixelInTile)};

if (windowEnabled && !inWindow)
    groupEnd = std::min(groupEnd, windowStart);
```

Tile edge. Batch end. Window start.

Decode pixels from those two bytes

```
int bitShift{TileWidth - 1 - pixelInTile};
do {
    std::uint8_t const colorIndex{static_cast<std::uint8_t>(
        ((row[0] >> bitShift) & 1) |
        (((row[1] >> bitShift) & 1) << 1))};
    state.backgroundColors[static_cast<std::size_t>(x)] =
        colorIndex;
    putPixel(x, y,
        static_cast<std::uint8_t>((BGP >> (2 * colorIndex)) & 3));
    --bitShift;
} while (++x < groupEnd);
```

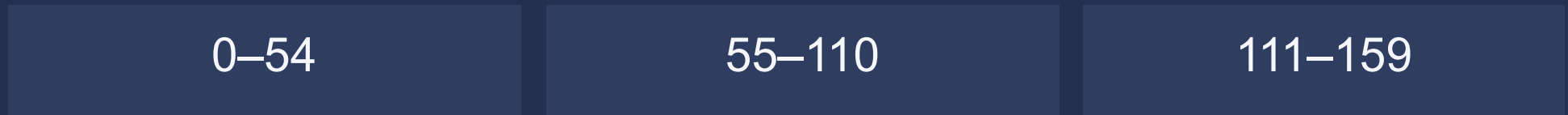
The first group might have only three pixels



SCX starts here →

5 6 7

The three existing rendering batches stay intact:



Signed tile indices still mean signed indices

```
int const tileIndex{
    usesSignedTileIndices
        ? static_cast<std::int8_t>(tileByte)
        : int{tileByte}};

auto const tileBase{
    usesSignedTileIndices ? SignedTileDataOffset : 0};
```

196,608 batch comparisons; sprites were left alone.

Now both builds are below one second

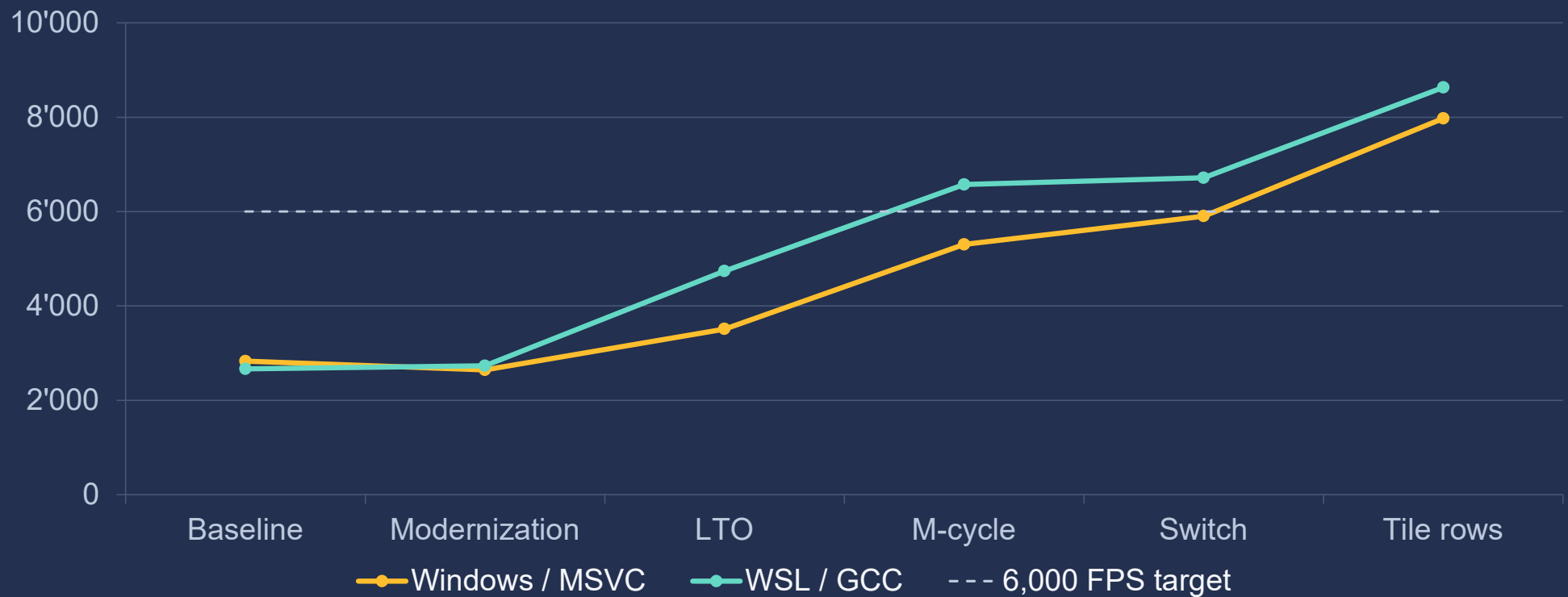
	Before FPS	After FPS	Change
Windows	5,903.5	7,975.7	+35.1%
WSL	6,717.6	8,631.8	+28.5%

133.5×

144.5×

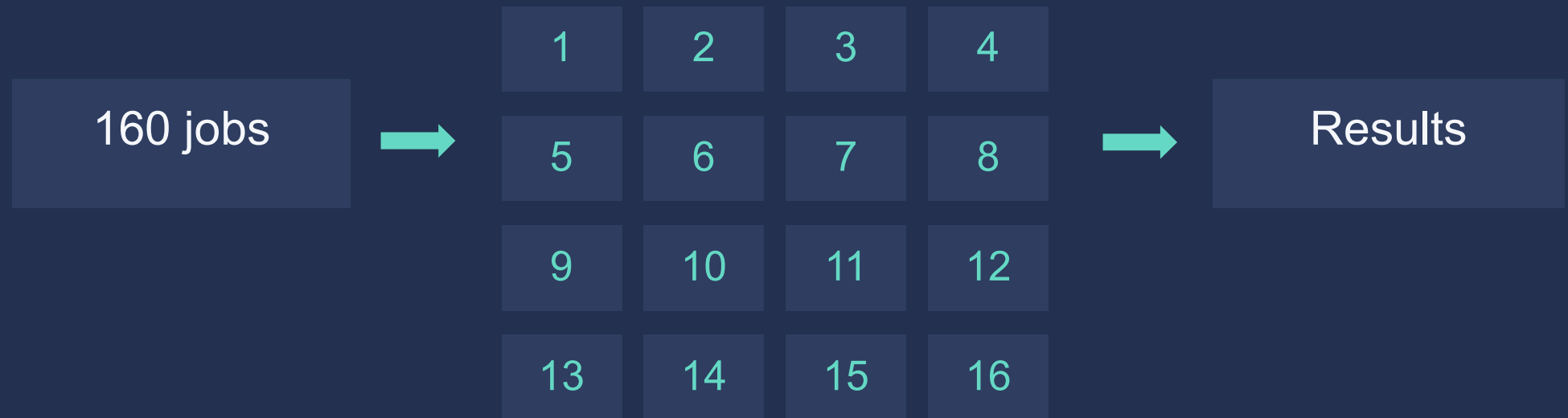
Cost: more boundary logic. No persistent cache to invalidate.

Where the five changes took us



2.82× baseline throughput on Windows; 3.24× on WSL.

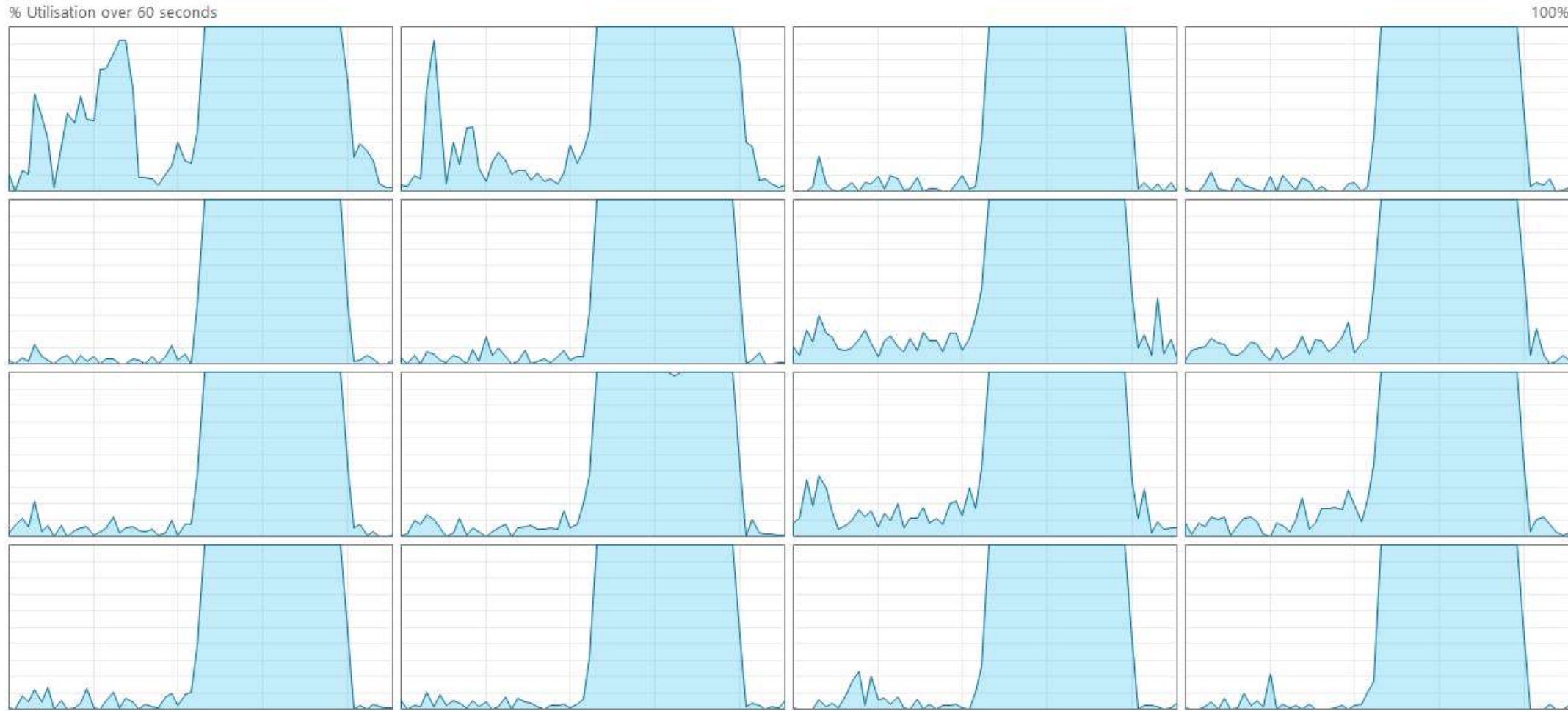
Independent emulators, one work queue



16 workers shown. Each worker owns an independent emulator.

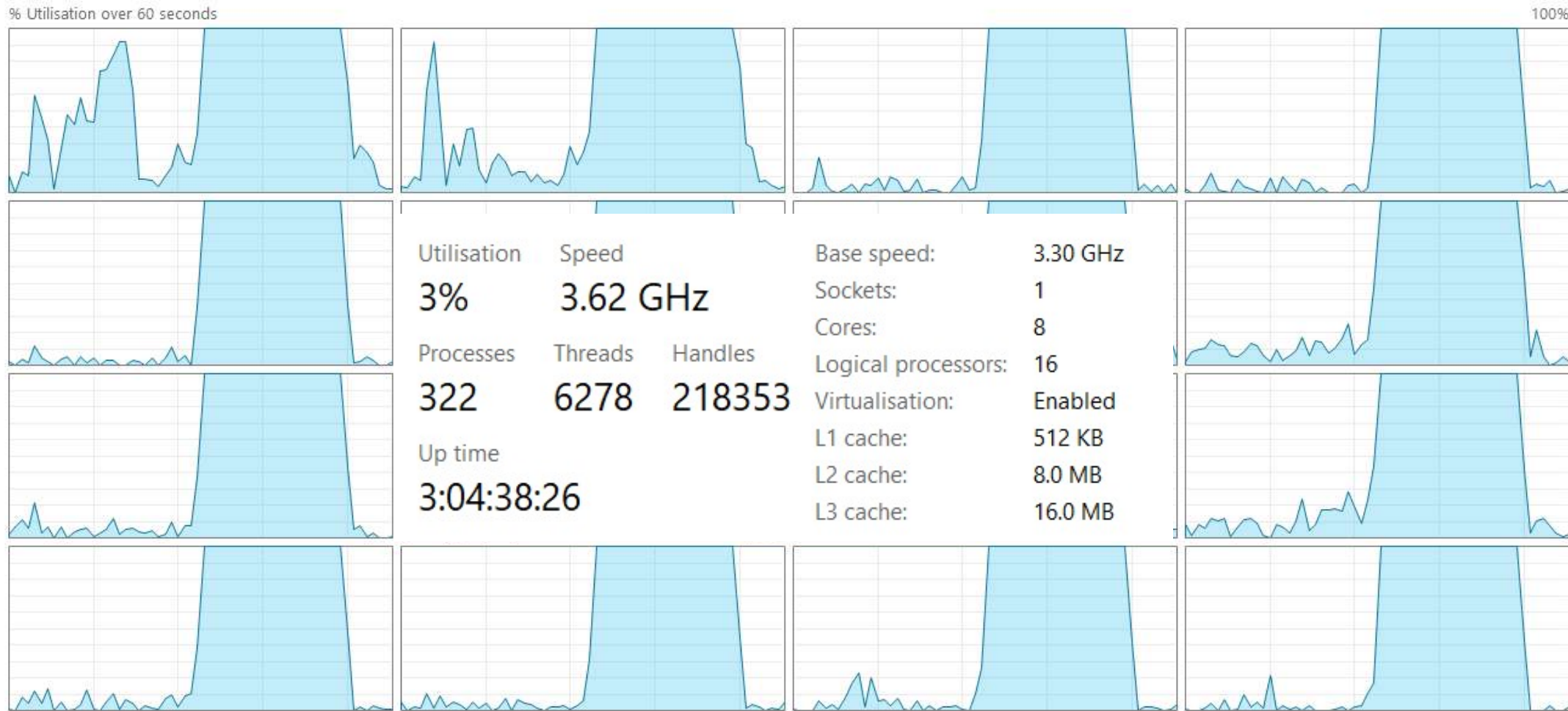
CPU

AMD Ryzen 7 250 w/ Radeon 780M Graphics



CPU

AMD Ryzen 7 250 w/ Radeon 780M Graphics



Each worker takes the next job

```
std::atomic<std::size_t> next{};

// Inside each worker:
for (;;) {
    auto const index{
        next.fetch_add(1, std::memory_order_relaxed)};
    if (index >= experiments.size())
        break;
    runExperiment(experiments[index], rom, recording, budget);
}
```

Now vary only the worker count

1–32

worker threads

160

replays at each count

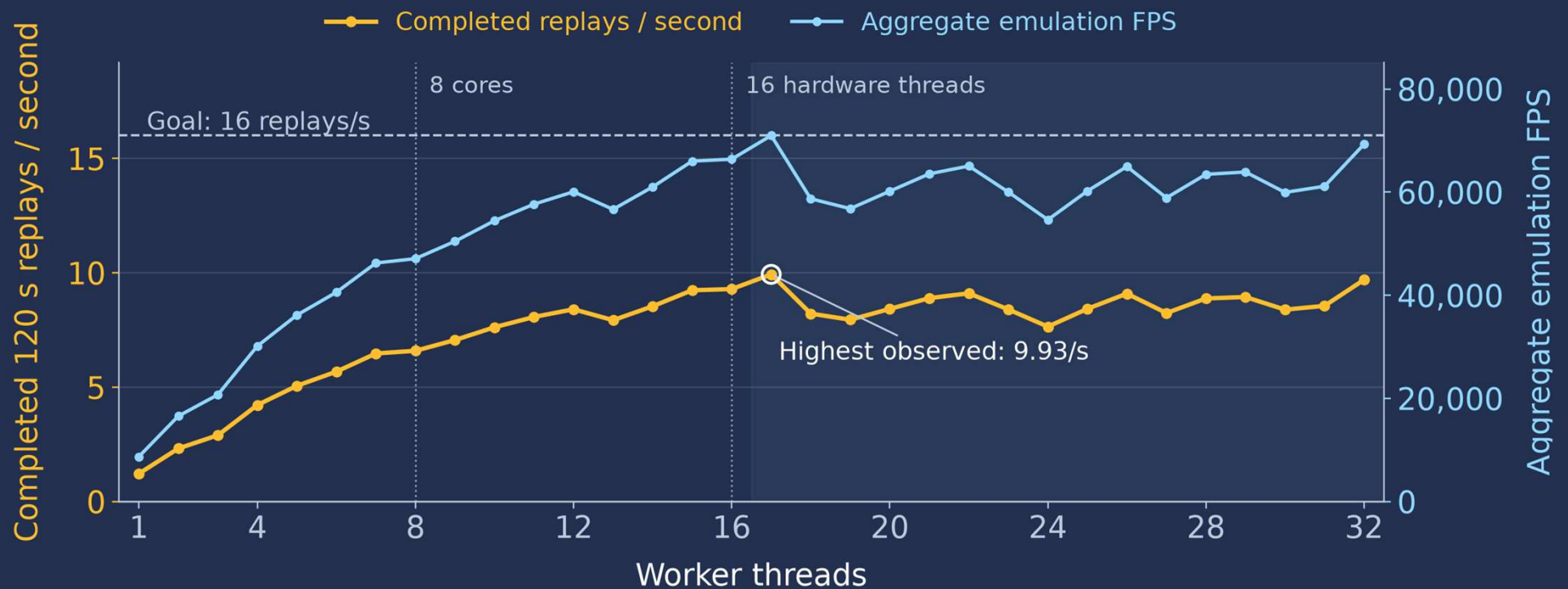
WSL • Ryzen 7 250 • 8 cores / 16 hardware threads

Same core, recording and cycle budget. Shuffled measurement order.

New recording after the RET fix; earlier slides retain their saved results.

More workers help—up to a point

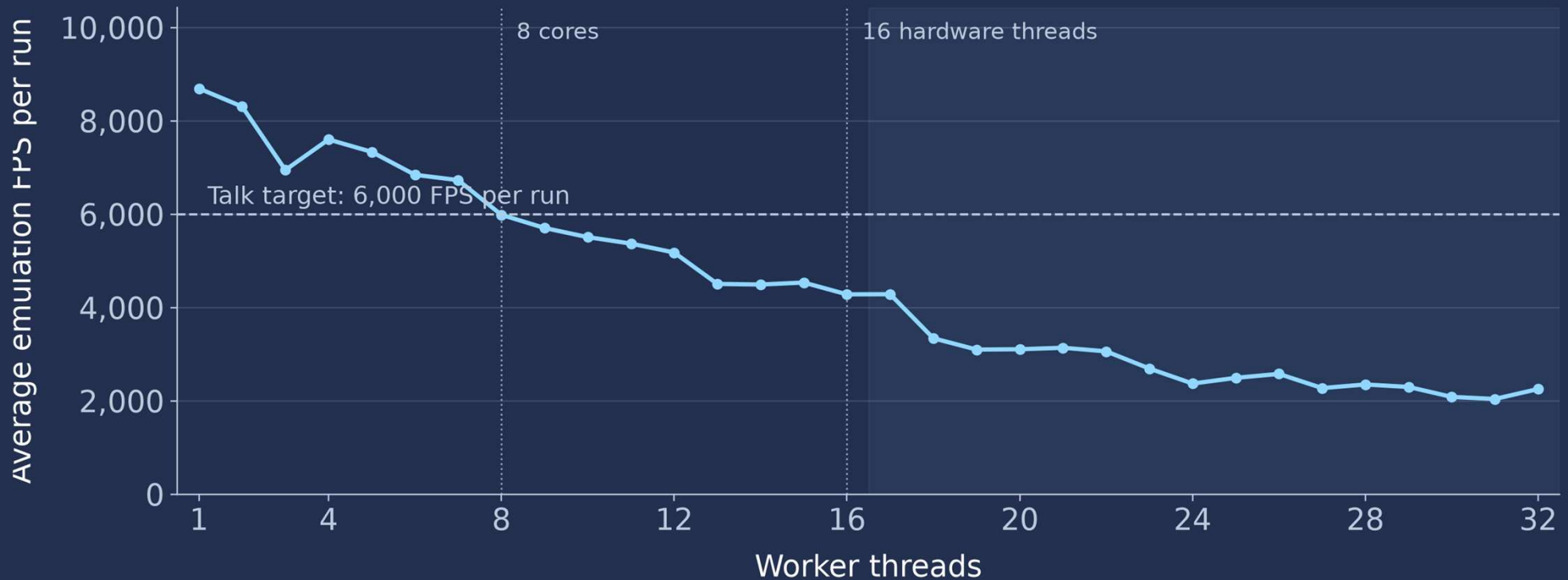
Batch throughput: replays/s and aggregate emulation FPS



Highest observed: 9.93 replays/s = 70,977 aggregate FPS at 17 workers • 8.17× one-worker throughput

Each replay slows as workers compete

Per-run FPS = all completed frames ÷ sum of individual run times



Mean run time: 822.4 ms at 1 worker → 1669.8 ms at 16 → 3172.6 ms at 32

Two hardware threads share a core

One physical core

Thread A

Thread B

Shared execution resources

16 workers

≠

16× throughput

The timings do not tell us which shared resource limited us.

6,000 FPS was only the first target

8,691 FPS

One replay at a time

9.93 replays/s

Highest observed: 17 workers

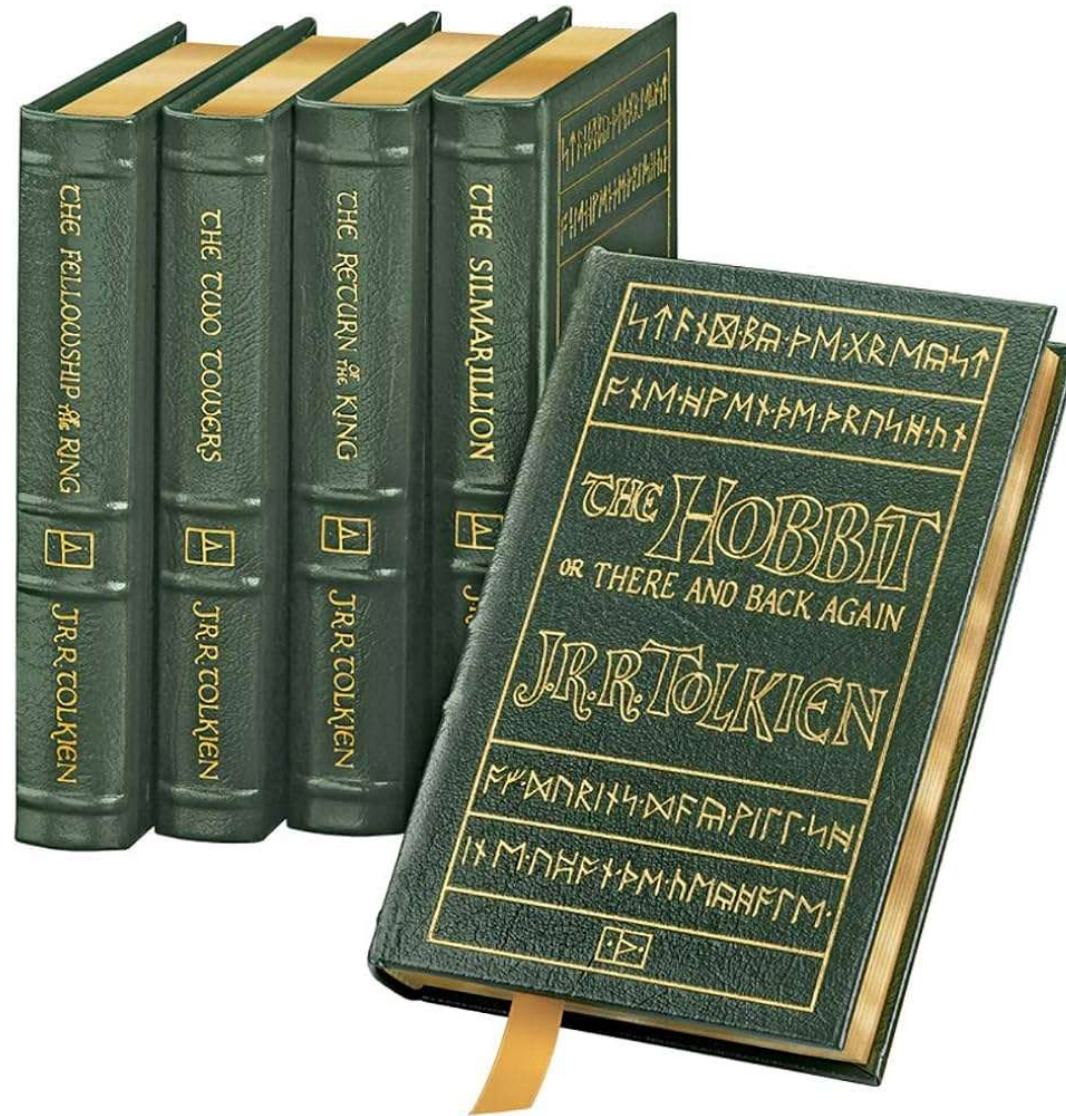
C++ changes accelerated one emulator.

The worker sweep exposed the throughput/latency trade-off.

The next question is which shared resource limits this workload.

Are we there yet?





The Nerd Snipers

With friends like these...



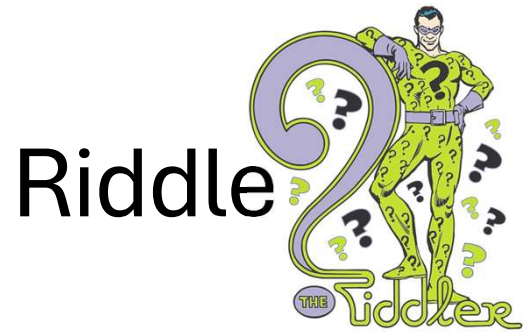
Tom Tesch



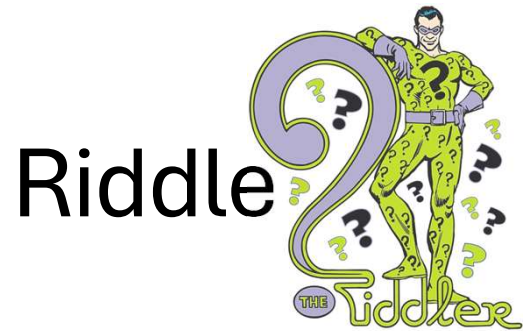
In Pursuit of a 6,000 FPS Game Boy Emulator



DEMO

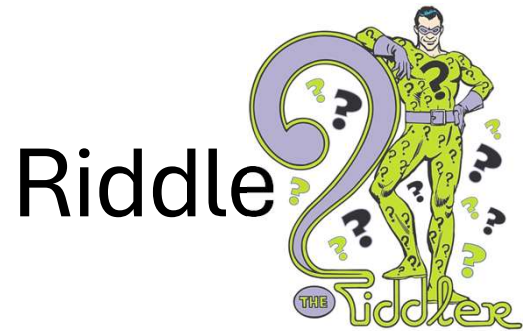


Why is **std::jthread** like the **--replay-CR** parameter of the Gameboy app?



Why is **std::jthread** like the **--replay-CR** parameter of the Gameboy app?

- Officially **std::jthread** are named for “joining thread”
- And **--replay-CR** is short for “Comic Replay”

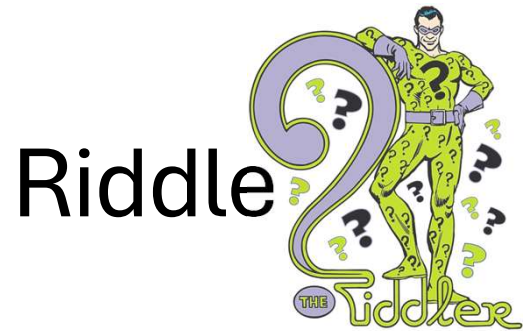


Riddle

Why is **std::jthread** like the **--replay-CR** parameter of the Gameboy app?

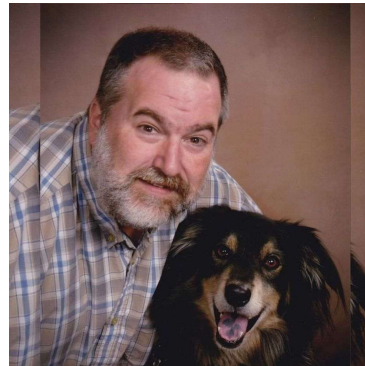
- Officially **std::jthread** are named for “joining threads”
- And **--replay-CR** is short for “Comic Replay”





Why is **std::jthread** like the **--replay-CR** parameter of the Gameboy app?

- Officially **std::jthread** are named for “joining threads”
- And **--replay-CR** is short for “Comic Replay”

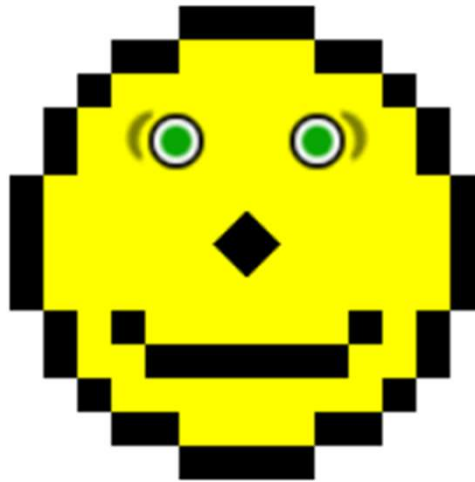




DEMO

Thanks Jonathan Stein 😊

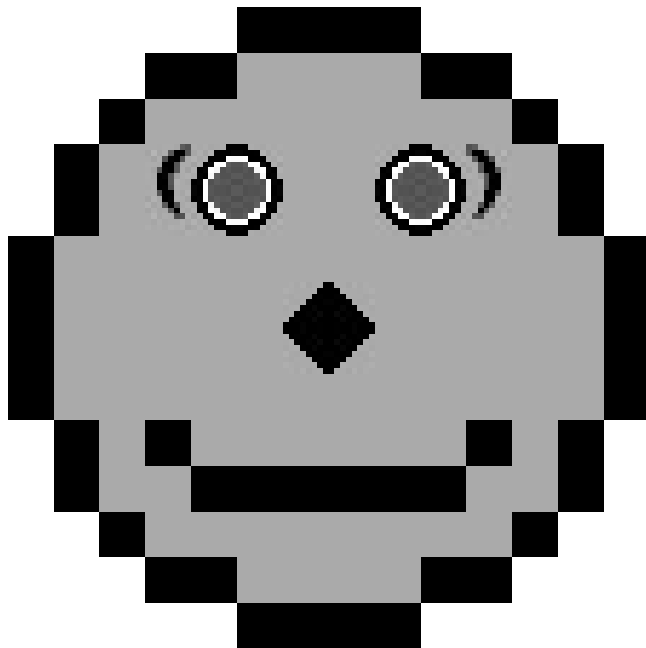
Hello World!



Source: <http://acid2.acidtests.org/reference.html>

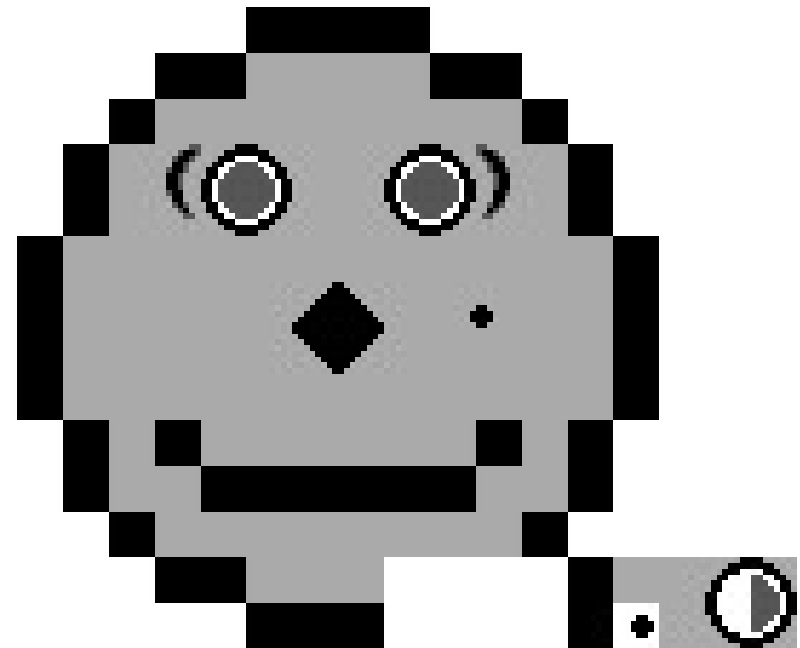
Thanks Jonathan Stein 😊

HELLO WORLD!



dmg-acid2 by Matt Currie

HELLO WORLD!



dmg-acid2 by Matt Currie



Graphics Programming with SDL 3 Part 2

📅 Wednesday September 16, 2026 16:45 - 17:45 MDT

📍 Homestead 3/4

Last year I introduced the SDL3 library which provides a solution for building games and graphical applications on multiple platforms. In this talk, we will continue our journey into SDL3, which newly includes the ability to write shaders within the built-in 2D API. My goal is to get you up and running fast by building out a small game application. Throughout the talk I will show how to build a small 2D framework, discussing design decisions and which C++ features can be used to help organize our code. Attendees will leave this talk ready to build multimedia / game applications and with an understanding on if SDL3 is the right tool for them.

Provide Feedback

Presenters



Mike Shah

Professor / (occasional) 3D Graphics Engineer

Mike Shah is currently a teaching faculty at Yale University with primary teaching interests in computer systems, computer graphics, and game engines. Mike's research interests are related to performance engineering (dynamic analysis), software visualization, and computer graphics... [Read More →](#)

● [GameDev](#)

Subject [Software Design](#), Tags [Breakouts](#)



RenderDoc

Latest Release: v1.46 - 31 Aug, 2026

Download (Win x64) ▾

[Source Code](#) • [Documentation](#) • [Other builds](#) • [Contact](#)



RenderDoc is a free MIT licensed stand-alone graphics debugger that allows quick and easy single-frame capture and detailed introspection of any application using Vulkan, D3D11, OpenGL & OpenGL ES or D3D12 across Windows, Linux, Android, or Nintendo Switch™.

Presenters



Mike Shah

Professor / (occasional) 3D Graphics Engineer

Mike Shah is currently a teaching faculty at Yale University with primary teaching interests in computer systems, computer graphics, and game engines. Mike's research interests are related to performance engineering (dynamic analysis), software visualization, and computer graphics... [Read More →](#)

● [GameDev](#)

Subject [Software Design](#), Tags [Breakouts](#)



[[musttail]]-ling Our Way to a Faster Python Interpreter and New JIT Compiler

📅 Thursday September 17, 2026 09:35 - 10:05 MDT

📍 Homestead 3/4

In CPython 3.14, we introduced an alternative interpreter implementation. This implementation used [[musttail]], an experimental C/C++ feature that tells the compiler to enforce tail calls. Against the previous interpreter using computed gotos/switch-case, the new interpreter style shows 2%--15% performance improvement (i.e. geometric mean running time), and is more resilient against certain types of compiler bugs. This feature is already supported by Clang, GCC, and MSVC, and is also the center of a draft proposal targeting C++29. In this talk, I'll cover the story of getting this feature into the Python interpreter, our collaboration with compiler developers, and the potential wider impact on the C++ community. I'll also cover interesting use cases of this feature other than for writing interpreters, such as for automatically generating a simple Just in Time (JIT) compiler.

Tooling Track sessions are sponsored by [Optiver](#).

Presenters



[Ken Jin Ooi](#)

Ken Jin Ooi is a Python maintainer since 2021 and is also currently a contractor for OpenAI. His Python work revolves around the performance of the Python interpreter, which is written in a subset of C11 that is mostly compatible with C++.

● [Tooling & Ecosystem](#)

Subject [Future C++](#), [Performance](#), Tags [Breakouts](#)

<https://www.iocccc.org/2025/ncw1/index.html>

```
#define G t[6]
#define A t[7]
#define M Z(2,0)
#define E (G&128)
#define B (G&16)
#define O if(y
#define Y O ==
#define _(r) (b=l(r++),b|l(r++)<<8)
#define S_(F)

typedef unsigned char u; u w,X,T,D[1<<16],t[]=R,U=255; unsigned
short J=-2,F=256,H=65280,b,l(a){ int r=D[a]; return a^H?r:(r&16?U>>4
: U)&15|r&240; } N(a,v){ if(a>>15)for(D[a]=a^H+68?v:0,a=(a==H+70)*160; a--;
) D[ 65024|a]=D[v<<8|a]; } Z(k,p){ return k<3?t[k*2]<<8|t[k*2+ 1]:p
?A<< 8|G:J; } P(k,v,p){ k<3?t[k*2]=v>>8,t[k*2+1]=v:p?(A=v>>8 ,G=v
&240 ): (J=v); } W(i){ return i^6?t[i]:l(M); } V(i,v){ i^6?t[i] =v:N
(M,v ); } L(z,n,c){ G=(z?128:0)|(n?64:0)|(c?16:0); } Q(v){ N( --J,
v>>8 ); N(--J,v); } j(o,v){ int c=(o&1 and o<4)*(G>>4&1),e=o>1 , h=
o &4 and o^7,r=h?(o&1?A^v:o>4?A|v:A&v):(e?A-v-c:A+v+c); G=(l(r &255
)<<7 )|((lh and e)<<6)|((lh and(e?r<0:r>255))<<4; if(o^7)A=r; } m({
if(X ) { if(D[65535]&D[H+15])X=0; return; } if(T){ w=1; T=0; } { u
o=C, x=o>>6,y=o>>3&7,z=o&7,p=y>>1,q=y&1; int a,c,e,r,v,k[]={ ! E,E,
!B,B } ; switch(x){ case 0:switch(z){ case 0:Y 2 or!y)return; Y 1
```

```
{ a= S; N(a,J); N(a+1,J>>8); return; } { e=(signed char)C; Y 3 or k
[y-4 ] )F+=e; return; } case 1:if(!q)P(p,S,0); else{ L(E,0,(r= M +Z
(p, 0 ))>65535); P(2,r,0); } return; case 2:a=p>1?M:Z(p,0); q ? (A
= l( a )):N(a,A); if(p>1)P(2,M+5-2*p,0); return; case 3:P(p,Z( p,0
)+1 - 2*q,0); return; case 4:case 5:o=W(y)+9-2*z; V(y,o); L(! o ,z
& 1, B ); return; case 6:V(y,C); return; default:O<4){ c=(y&1? A :A
>> 7 ) &1; A=y&1?A>>1:A<<1; A or_eq(y&2? !B:c)<<((y&1)*7); L( 0 ,0
,c ) ; } else Y 4){ if(G&64){ if(B)A-=96; } else{ if(B|A>153) { A
+=96 ; G or_eq 16; } if((A&15)>9)A+=6; } L(!A,G&64,B); } else Y 5)
{ A = ~A; L(E,1,B); } else{ L(E,0,y&1? !B:1); } return; } case 1
:if( o ==118)X=1; else V(y,W(z)); return; case 2:j(y,W(z)) ; return
; default :switch(z){ case 0:O<4){ if(k[y])F=_J); return ; }
Y 4) { N(H|C,A);
return; } Y 6){ A=l(H|C); return; } { e=(signed char)C; r=J+e; L(0,0,
(J^e^r)&256); Y 5)J=r; else P(2,r,0); return; } case 1:if(!q){ P(p,_
J ),1); return; } if(p&2){ p&1?(J=M):(F=M); return; } F=_J); w or_eq
p ; return; case 2:a=(y<4 or y&1)?S:H|t[1]; O<4){ if(k[y])F=a; } else
{ y&2?(A=l(a)):N(a,A); } return; case 3:if(!y){ a=S; F=a; return; } O
>5){ y==6?(w=T=0):(T=1); return; } o=C; x=o>>6; y=o>>3&7; z=o&7; v=r=
W(z); if( x==1){ L(!v&(1<<y)),0,B); return; } if(x)r = x==2?v
& ~(1<<y ) :v|1<<y; else{ c=y not_eq 6&v>>7* !(y&1) ; r=y==
6?v<< 4|v>>4:(y&1?v>>1:v<<1); O<4)r or_eq(y&2 ? !B:
c)<< (y&1)*7; Y 5)r or_eq v&128; x=r ; L( ! x,0,c)
; } V (z,r); return; case 4:O<4){ a=S ; if(k[y]){ Q(
```

```
F); F=a ; } } return; case 5:if(!q)Q(Z(p,1 )); else if(!p)
{ a=S; Q ( F); F=a; } return; case 6:j(y,C); return ; default
: Q(F); F=y*8; return; } } } i(x,y){ int e=D[H+64],c=0,p,q,h,k=0; u
* o; if(e&1){ o=D+H+66; p=x+o[1]; q=y+ *o; h=D[38912+(q/8&31)*32+(p/8
& 31)]; o=D+(e&16?8:9)*4096+(e&16?h:(signed char)h)*16+(q&7)*2; c=( *o
>>(7-(p&7))&1)|(o[1]>>(7-(p&7))&1)*2; } if(e&2)for( k<40; k++) { o=D+
65024+k*4; q=y- *o+16; p=x-o[1]+8; if((unsigned)q<8 and(unsigned)p<8)
{ o=D+32768+o[2]*16+q*2; h=( *o>>(7-p)&1)|(o[1]>>(7-p)&1)*2; if(h)c=h
; } } return 255-7*(e & 128?c:0) ; } main(){ static u d[9]; u*
a=D+H+68,c; char*k= K,*x,*e,s[ 1<<20]; int p=0,q =0,f ,g; read(
3,D,32768); D[H] = 207; a[- 4 ]=145; a[187] = 1; for( ; ; ){
m(); D[H+4]++; p+=8; for( p>=456; p-=456) ++ * a== 144? D[H+15
] or_eq 1,q=0: *a>153? *a=q=0:0; if(*a==144 and ! q) { q = 1;
while(read(0,&c,1)>0)x=strchr(k,c),x?d[x-k]= 9:0; for ( U= 255
, f =0; f<8; f++)d[f]?d[f]--,U xor_eq 1<<f:0; for ( e= strcpy(
s,"\\x1b[H")+3,f=0; f<144; *e++=10,f+=2)for(g= 0; g <160; g++)
e+=sprintf(e,"\\x1b[38;5;%dm\\x1b[48;5;%dm",i(g, f),i(g,f+1
)); write(1,s,e-s); usleep(14000); } w and(D[65535]&D
[H+15]&1)?w=0,D[H+15]and_eq~1,Q(F),F=64:0; } }
```

<https://www.ioccc.org/2025/ncw1/index.html>

```
#define G t[6]
#define A t[7]
#define M Z(2,0)
#define E (G&128)
#define B (G&16)
#define O if(y
#define Y O ==
#define _(r) (b=l(r++),b|l(r++)<<8)
#define S_(F)

typedef unsigned char u; u w,X,T,D[1<<16],t[]=R,U=255; unsigned
short J=-2,F=256,H=65280,b;l(a){ int r=D[a]; return a^H?r:(r&16?U>>4
: U)&
15|r&
240; } N(a,v){ if(a>>15)for(D[a]=a^H+68?v:0,a=(a==H+70)*160; a--;
) D[ 65024|a]=D[v<<8|a]; } Z(k,p){ return k<3?t[k*2]<<8|t[k*2+ 1]:p
?A<< 8|G:J; } P(k,v,p){ k<3?t[k*2]=v>>8,t[k*2+1]=v:p?(A=v>>8 ,G=v
&240 ): (J=v); } W(i){ return i^6?t[i]:l(M); } V(i,v){ i^6?t[i] =v:N
(M,v ); } L(z,n,c){ G=(z?128:0)|(n?64:0)|(c?16:0); } Q(v){ N( --J,
v>>8 ); N(--J,v); } j(o,v){ int c=(o&1 and o<4)*(G>>4&1),e=o>1 , h=
o &4 and o^7,r=h?(o&1?A^o:v:o>4?A|v:A&v):(e?A-v-c:A+v+c); G=(l(r &255
)<<7 )|((!h and e)<<6)|(!h and (e?r<0:r>255))<<4; if(o^7)A=r; } m(){
if(X ) { if(D[65535]&D[H+15])X=0; return; } if(T){ w=1; T=0; } { u
o=C, x=o>>6,y=o>>3&7,z=o&7,p=y>>1,q=y&1; int a,c,e,r,v,k[ ]={ ! E,E,
!B,B }; switch(x){ case 0:switch(z){ case 0:Y 2 or!y)return; Y 1
```

```
{ a= S; N(a,J); N(a+1,J>>8); return; } { e=(signed char)C; Y 3 or k
[y-4 ] )F+=e; return; } case 1:if(!q)P(p,S,0); else{ L(E,0,(r= M +Z
(p, 0 ))>65535); P(2,r,0); } return; case 2:a=p>1?M:Z(p,0); q ? (A
= l( a )):N(a,A); if(p>1)P(2,M+5-2*p,0); return; case 3:P(p,Z( p,0
)+1 - 2*q,0); return; case 4:case 5:o=W(y)+9-2*z; V(y,o); L(! o ,z
& 1, B ); return; case 6:V(y,C); return; default:O<4){ c=(y&1? A :A
>> 7 ) &1; A=y&1?A>>1:A<<1; A or_eq(y&2? !B:c)<<((y&1)*7); L( 0 ,0
,c ) ; } else Y 4){ if(G&64){ if(B)A-=96; } else{ if(B|A>153) { A
+=96 ; G or_eq 16; } if((A&15)>9)A+=6; } L(!A,G&64,B); } else Y 5)
{ A = ~A; L(E,1,B); } else{ L(E,0,y&1? !B:1); } return; } case 1
:if( o ==118)X=1; else V(y,W(z)); return; case 2:j(y,W(z)) ; return
; default :switch(z){ case 0:O<4){ if(k[y])F=_J); return ; }
Y 4) { N(H|C,A);
return; } Y 6){ A=l(H|C); return; } { e=(signed char)C; r=J+e; L(0,0,
(J^e^r)&256); Y 5)J=r; else P(2,r,0); return; } case 1:if(!q){ P(p,_
J ),1); return; } if(p&2){ p&1?(J=M):(F=M); return; } F=_J); w or_eq
p ; return; case 2:a=(y<4 or y&1)?S:H|t[1]; O<4){ if(k[y])F=a; } else
{ y&2?(A=l(a)):N(a,A); } return; case 3:if(!y){ a=S; F=a; return; } O
>5){ y==6?(w=T=0):(T=1); return; } o=C; x=o>>6; y=o>>3&7; z=o&7; v=r=
W(z); if( x==1){ L(!(v&(1<<y)),0,B); return; } if(x)r = x==2?v
& ~(1<<y ) :v|1<<y; else{ c=y not_e
6?v<< 4|v>>4:(y&1?v>>1:v<<1);
c)<< (y&1)*7; Y 5)r or_eq v&12;
; } V (z,r); return; case 4:O<4){ a
```

```
F); F=a ; } } return; case 5:if(!q)Q(Z(p,1 )); else if(!p)
{ a=S; Q (F); F=a; } return; case 6:j(y,C); return ; default
: Q(F); F=y*8; return; } } { i(x,y){ int e=D[H+64],c=0,p,q,h,k=0; u
* o; if(e&1){ o=D+H+66; p=x+o[1]; q=y+ *o; h=D[38912+(q/8&31)*32+(p/8
& 31)]; o=D+(e&16?8:9)*4096+(e&16?h:(signed char)h)*16+(q&7)*2; c=(*o
>>(7-(p&7))&1)|(o[1]>>(7-(p&7))&1)*2; } if(e&2)for( k<40; k++){ o=D+
65024+k*4; q=y- *o+16; p=x-o[1]+8; if(((unsigned)q<8 and (unsigned)p<8)
{ o=D+32768+o[2]*16+q*2; h=(*o>>(7-p))&1)|(o[1]>>(7-p)&1)*2; if(h)c=h
; } } return 255-7*(e & 128?c:0) ; } main(){ static u d[9]; u*
a=D+H+68,c; char*k= K,*x,*e,s[ 1<<20]; int p=0,q =0,f ,g; read(
3,D,32768); D[H] = 207; a[- 4]=145; a[187] = 1; for( ; ; ){
m(); D[H+4]++; p+=8; for( p>=456; p-=456) ++ * a == 144? D[H+15
] or_eq 1,q=0: *a>153? *a=q=0:0; if(*a==144 and ! q) { q = 1;
while(read(0,&c,1)>0)x=strchr(k,c),x?d[x-k]= 9:0; for ( U= 255
, f =0; f<8; f++)d[f]?d[f]--,U xor_eq 1<<f:0; for ( e= strcpy(
s,"\x1b[H")+3,f=0; f<144; *e++ =10,f+=2)for(g= 0; g <160; g++)
e+=sprintf(e,"\x1b[38;5;%dm\x1b[48;5;%dm",i(g, f),i(g,f+1
)); write(1,s,e-s); usleep(14000); } w and(D[65535]&D
[H+15]&1)?w=0,D[H+15]and_eq~1,Q(F),F=64:0; } }
```

```
, f =0; f<8; f++)d[f]?d[f]--,U xor_eq 1<<f:0; for ( e= strcpy(
s,"\x1b[H")+3,f=0; f<144; *e++ =10,f+=2)for(g= 0; g <160; g++)
e+=sprintf(e,"\x1b[38;5;%dm\x1b[48;5;%dm",i(g, f),i(g,f+1
)); write(1,s,e-s); usleep(14000); } w and(D[65535]&D
[H+15]&1)?w=0,D[H+15]and_eq~1,Q(F),F=64:0; } }
```


Questions?

Repo →



```
1  import std;
2
3  int main() {
4      for (char const code :
5          std::string_view{R"(z/:$zb$k{!$W{I$H{g$4{{0$%vWv$lz b'$bz4I6)"
6                          R"($XzH+J$SW:RY$NMX/m$IHg|$IHST%6%, $ICiY9"9)"
7                          R"("$IH5<T%6%, $IH!d|$NMZ/m$SW<RY$I-zH-J$5Kz)"
8                          R"(4K6$&dz d'$%xWx$4}}2$H}i$W}K$k}#$zd$z/<$)"})" {
9          int const value{code - ' '};
10         std::print("{} ", std::string(value / 5 + 1, " .=#\n"[value % 5]));
11     }
12     std::cin.get();
13 }
```

Answer!

Repo →

